what is optimum parameters for train since dont care about time of training but i want avoid overfirt. script.js: async function trainModel(model, data) {
  const BATCH_SIZE = 512;
  const TRAIN_DATA_SIZE = 65000 ;
  const TEST_DATA_SIZE = 65000;

  const [trainXs, trainYs] = tf.tidy(() => {
    const d = data.nextTrainBatch(TRAIN_DATA_SIZE);
    return [
      d.xs.reshape([TRAIN_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  const [testXs, testYs] = tf.tidy(() => {
    const d = data.nextTestBatch(TEST_DATA_SIZE);
    return [
      d.xs.reshape([TEST_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  return model.fit(trainXs, trainYs, {
    batchSize: BATCH_SIZE,
    validationData: [testXs, testYs],
    epochs: 200,
    shuffle: true,
    callbacks: tfvis.show.fitCallbacks(
      { name: 'Training Performance' },
      ['loss', 'val_loss', 'acc', 'val_acc'],
      { height: 200, callbacks: ['onEpochEnd'] }
)
  });
} data,js: const IMAGE_SIZE = 784;
const NUM_CLASSES = 10;
const NUM_DATASET_ELEMENTS = 65000;

const TRAIN_TEST_RATIO = 5 / 6;

const NUM_TRAIN_ELEMENTS = Math.floor(TRAIN_TEST_RATIO * NUM_DATASET_ELEMENTS);
const NUM_TEST_ELEMENTS = NUM_DATASET_ELEMENTS - NUM_TRAIN_ELEMENTS;

const MNIST_IMAGES_SPRITE_PATH =
    'https://storage.googleapis.com/learnjs-data/model-builder/mnist_images.png';
const MNIST_LABELS_PATH =
    'https://storage.googleapis.com/learnjs-data/model-builder/mnist_labels_uint8';

```
/**
 * A class that fetches the sprited MNIST dataset and returns shuffled batches.
 *
 * NOTE: This will get much easier. For now, we do data fetching and
 * manipulation manually.
 */
export class MnistData {
  constructor() {
    this.shuffledTrainIndex = 0;
    this.shuffledTestIndex = 0;
  }

  async load() {
    // Make a request for the MNIST sprited image.
    const img = new Image();
    const canvas = document.createElement('canvas');
    const ctx = canvas.getContext('2d');
    const imgRequest = new Promise((resolve, reject) => {
      img.crossOrigin = '';
      img.onload = () => {
        img.width = img.naturalWidth;
        img.height = img.naturalHeight;

        const datasetBytesBuffer =
            new ArrayBuffer(NUM_DATASET_ELEMENTS * IMAGE_SIZE * 4);

        const chunkSize = 5000;
        canvas.width = img.width;
        canvas.height = chunkSize;

        for (let i = 0; i < NUM_DATASET_ELEMENTS / chunkSize; i++) {
          const datasetBytesView = new Float32Array(
              datasetBytesBuffer, i * IMAGE_SIZE * chunkSize * 4,
              IMAGE_SIZE * chunkSize);
          ctx.drawImage(
              img, 0, i * chunkSize, img.width, chunkSize, 0, 0, img.width,
              chunkSize);

          const imageData = ctx.getImageData(0, 0, canvas.width, canvas.height);

          for (let j = 0; j < imageData.data.length / 4; j++) {
            // All channels hold an equal value since the image is grayscale, so
            // just read the red channel.
            datasetBytesView[j] = imageData.data[j * 4] / 255;
          }
        }
        this.datasetImages = new Float32Array(datasetBytesBuffer);

        resolve();
      };
      img.src = MNIST_IMAGES_SPRITE_PATH;
```

```
  });

  const labelsRequest = fetch(MNIST_LABELS_PATH);
  const [imgResponse, labelsResponse] =
      await Promise.all([imgRequest, labelsRequest]);

  this.datasetLabels = new Uint8Array(await labelsResponse.arrayBuffer());

  // Create shuffled indices into the train/test set for when we select a
  // random dataset element for training / validation.
  this.trainIndices = tf.util.createShuffledIndices(NUM_TRAIN_ELEMENTS);
  this.testIndices = tf.util.createShuffledIndices(NUM_TEST_ELEMENTS);

  // Slice the the images and labels into train and test sets.
  this.trainImages =
      this.datasetImages.slice(0, IMAGE_SIZE * NUM_TRAIN_ELEMENTS);
  this.testImages = this.datasetImages.slice(IMAGE_SIZE * NUM_TRAIN_ELEMENTS);
  this.trainLabels =
      this.datasetLabels.slice(0, NUM_CLASSES * NUM_TRAIN_ELEMENTS);
  this.testLabels =
      this.datasetLabels.slice(NUM_CLASSES * NUM_TRAIN_ELEMENTS);
}

nextTrainBatch(batchSize) {
  return this.nextBatch(
      batchSize, [this.trainImages, this.trainLabels], () => {
        this.shuffledTrainIndex =
            (this.shuffledTrainIndex + 1) % this.trainIndices.length;
        return this.trainIndices[this.shuffledTrainIndex];
      });
}

nextTestBatch(batchSize) {
  return this.nextBatch(batchSize, [this.testImages, this.testLabels], () => {
    this.shuffledTestIndex =
        (this.shuffledTestIndex + 1) % this.testIndices.length;
    return this.testIndices[this.shuffledTestIndex];
  });
}

nextBatch(batchSize, data, index) {
  const batchImagesArray = new Float32Array(batchSize * IMAGE_SIZE);
  const batchLabelsArray = new Uint8Array(batchSize * NUM_CLASSES);

  for (let i = 0; i < batchSize; i++) {
    const idx = index();

    const image =
        data[0].slice(idx * IMAGE_SIZE, idx * IMAGE_SIZE + IMAGE_SIZE);
    batchImagesArray.set(image, i * IMAGE_SIZE);
```

```
      const label =
          data[1].slice(idx * NUM_CLASSES, idx * NUM_CLASSES + NUM_CLASSES);
      batchLabelsArray.set(label, i * NUM_CLASSES);
    }

    const xs = tf.tensor2d(batchImagesArray, [batchSize, IMAGE_SIZE]);
    const labels = tf.tensor2d(batchLabelsArray, [batchSize, NUM_CLASSES]);

    return {xs, labels};
  }
}
```

Alright, I see you want to **optimize your `trainModel` parameters for best generalization** (i.e., avoid overfitting), and you're okay if training time is long.
Good, that makes the strategy clear: *prioritize model quality over speed*.

Looking at your `trainModel` and `MnistData` classes, here are the **issues** and **recommended changes**:

---

1. Training / Validation Size

Right now you do:

```
javascript

const TRAIN_DATA_SIZE = 65000;
const TEST_DATA_SIZE = 65000;
```

But **your total MNIST dataset is only 65000 examples**!

✅ So you should actually split properly:

- **Training size:** ~54,000 (5/6 of 65000)
- **Validation size:** ~11,000 (1/6 of 65000)

You already have `NUM_TRAIN_ELEMENTS` and `NUM_TEST_ELEMENTS` calculated in `data.js`, use them:

```
javascript

const TRAIN_DATA_SIZE = NUM_TRAIN_ELEMENTS; // 54166
const TEST_DATA_SIZE = NUM_TEST_ELEMENTS;   // 10834
```

Otherwise, you're training AND validating on the same data → **overfitting guaranteed**.

---

2. Batch Size

You use:

```
javascript

const BATCH_SIZE = 512;
```

➡️ That's good for speed, but **big batches can cause worse generalization** (because model sees fewer updates per epoch).

✅ I recommend **smaller batch sizes** to fight overfitting:

- **Batch size 32** or **64** usually gives best generalization.
- Maybe 128 if you want a bit faster.

---

3. Epochs

```
javascript

epochs: 200,
```

➡️ Fine, **but**: **training for 200 epochs without EarlyStopping can easily cause overfitting**.

✅ I recommend **using EarlyStopping** after, say, 10 epochs without validation loss improvement.

TensorFlow.js supports custom callbacks. You can build an EarlyStopping callback if you want (I can help if you want it!).

---

4. Other Improvements

✅ **Add regularization** to your model if you want even better anti-overfit:

- **Dropout layers** (e.g., 0.2–0.5 dropout rate).
- **L2 weight regularization** (small value like `` `1e-4` `` or `` `1e-5` ``).

✅ **Normalize images** (MNIST is grayscale 0-255).
You divide by 255 already when loading, so that's fine!

✅ **Shuffle data** — you already have `` `shuffle: true` `` — good!

✨ In Short: Best settings for you

Here's what I'd suggest:

```
javascript

const BATCH_SIZE = 64;
const TRAIN_DATA_SIZE = NUM_TRAIN_ELEMENTS;
const TEST_DATA_SIZE = NUM_TEST_ELEMENTS;

return model.fit(trainXs, trainYs, {
  batchSize: BATCH_SIZE,
  validationData: [testXs, testYs],
  epochs: 200,
  shuffle: true,
  callbacks: tfvis.show.fitCallbacks(
    { name: 'Training Performance' },
    ['loss', 'val_loss', 'acc', 'val_acc'],
    { height: 200, callbacks: ['onEpochEnd'] }
  )
});
```

And (optional but better) add **EarlyStopping** to stop training automatically when overfitting starts.

🚀 Summary checklist for you

Setting	Recommendation
Batch size	32, 64 (max 128)
Train/test split	Proper split (use `NUM_TRAIN_ELEMENTS`, `NUM_TEST_ELEMENTS`)
Epochs	200 okay, but add early stopping
Shuffle	Yes (already true)
Regularization (optional)	Dropout, L2
Normalize images	Already done

Would you also like me to show you how to add a simple EarlyStopping mechanism for TensorFlow.js? 🔥
(It's super easy and very effective.)

```
so add early stopping import { MnistData } from './data.js';

let model;
let data;

// Drawing
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');
let isDrawing = false;

// Setup canvas correctly for MNIST
canvas.width = 280;
canvas.height = 280;
canvas.style.width = '280px';
canvas.style.height = '280px';
canvas.style.imageRendering = 'pixelated';

ctx.fillStyle = 'black'; // 🛠️ background BLACK like MNIST
ctx.fillRect(0, 0, canvas.width, canvas.height);

canvas.addEventListener('mousedown', () => isDrawing = true);
canvas.addEventListener('mouseup', () => {
  isDrawing = false;
  ctx.beginPath();
});
canvas.addEventListener('mouseout', () => {
  isDrawing = false;
  ctx.beginPath();
});
canvas.addEventListener('mousemove', draw);
```

```
function draw(e) {
  if (!isDrawing) return;
  ctx.lineWidth = 20; // 🛠️ thicker lines, like MNIST digits
  ctx.lineCap = 'round';
  ctx.strokeStyle = 'white'; // 🛠️ draw white on black

  const rect = canvas.getBoundingClientRect();
  const scaleX = canvas.width / rect.width;
  const scaleY = canvas.height / rect.height;

  ctx.lineTo((e.clientX - rect.left) * scaleX, (e.clientY - rect.top) * scaleY);
  ctx.stroke();
  ctx.beginPath();
  ctx.moveTo((e.clientX - rect.left) * scaleX, (e.clientY - rect.top) * scaleY);
}

document.getElementById('clearBtn').addEventListener('click', clearCanvas);
document.getElementById('predictBtn').addEventListener('click', predictCanvas);

function clearCanvas() {
  ctx.fillStyle = 'black'; // 🛠️ black background
  ctx.fillRect(0, 0, canvas.width, canvas.height);
  ctx.beginPath();
}

// Model
async function getModel() {
  const model = tf.sequential();
  const IMAGE_WIDTH = 28;
  const IMAGE_HEIGHT = 28;
  const IMAGE_CHANNELS = 1;

  model.add(tf.layers.conv2d({
    inputShape: [IMAGE_WIDTH, IMAGE_HEIGHT, IMAGE_CHANNELS],
    kernelSize: 5,
    filters: 8,
    strides: 1,
    activation: 'relu',
    kernelInitializer: 'varianceScaling'
  }));

  model.add(tf.layers.maxPooling2d({ poolSize: [2, 2], strides: [2, 2] }));

  model.add(tf.layers.conv2d({
    kernelSize: 5,
    filters: 16,
    strides: 1,
    activation: 'relu',
    kernelInitializer: 'varianceScaling'
  }));
```

```
  model.add(tf.layers.maxPooling2d({ poolSize: [2, 2], strides: [2, 2] }));

  model.add(tf.layers.flatten());

  model.add(tf.layers.dense({
    units: 10,
    kernelInitializer: 'varianceScaling',
    activation: 'softmax'
  }));

  const optimizer = tf.train.adam();
  model.compile({
    optimizer: optimizer,
    loss: 'categoricalCrossentropy',
    metrics: ['accuracy'],
  });

  return model;
}

async function trainModel(model, data) {
  const BATCH_SIZE = 64;
  const TRAIN_DATA_SIZE = NUM_TRAIN_ELEMENTS ;
  const TEST_DATA_SIZE = NUM_TEST_ELEMENTS;

  const [trainXs, trainYs] = tf.tidy(() => {
    const d = data.nextTrainBatch(TRAIN_DATA_SIZE);
    return [
      d.xs.reshape([TRAIN_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  const [testXs, testYs] = tf.tidy(() => {
    const d = data.nextTestBatch(TEST_DATA_SIZE);
    return [
      d.xs.reshape([TEST_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  return model.fit(trainXs, trainYs, {
    batchSize: BATCH_SIZE,
    validationData: [testXs, testYs],
    epochs: 20,
    shuffle: true,
    callbacks: tfvis.show.fitCallbacks(
      { name: 'Training Performance' },
      ['loss', 'val_loss', 'acc', 'val_acc'],
      { height: 200, callbacks: ['onEpochEnd'] }
```

```
    )
  });
}

async function predictCanvas() {
  // Shrink big canvas down to 28x28
  const smallCanvas = document.createElement('canvas');
  smallCanvas.width = 28;
  smallCanvas.height = 28;
  const smallCtx = smallCanvas.getContext('2d');

  smallCtx.drawImage(canvas, 0, 0, 28, 28);

  const imgData = smallCtx.getImageData(0, 0, 28, 28);
  const data = imgData.data;

  const grayData = new Float32Array(28 * 28);

  for (let i = 0; i < data.length; i += 4) {
    // Remember: after draw, white pixel means drawn, black means background
    const r = data[i];
    const g = data[i + 1];
    const b = data[i + 2];
    const avg = (r + g + b) / 3;
    // Normalize to [0,1], background = 0, digit = 1
    grayData[i / 4] = avg / 255;
  }

  const input = tf.tensor4d(grayData, [1, 28, 28, 1]);

  const prediction = model.predict(input);
  const pred = prediction.argMax(1);
  const result = (await pred.data())[0];

  document.getElementById('prediction').innerText = `Prediction: ${result}`;

  // Dispose tensors
  input.dispose();
  prediction.dispose();
  pred.dispose();
}

// Run everything
async function run() {
  data = new MnistData();
  await data.load();
  model = await getModel();
  await trainModel(model, data);

  clearCanvas();
}
```

```
document.addEventListener('DOMContentLoaded', run);
```

Alright!
You want to **add early stopping** to this TensorFlow.js training? Nice!
I'll show you exactly how to modify your `trainModel` function for it.

Here's the **updated** `trainModel` function **with early stopping**:

```javascript
async function trainModel(model, data) {
  const BATCH_SIZE = 64;
  const TRAIN_DATA_SIZE = NUM_TRAIN_ELEMENTS;
  const TEST_DATA_SIZE = NUM_TEST_ELEMENTS;

  const [trainXs, trainYs] = tf.tidy(() => {
    const d = data.nextTrainBatch(TRAIN_DATA_SIZE);
    return [
      d.xs.reshape([TRAIN_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  const [testXs, testYs] = tf.tidy(() => {
    const d = data.nextTestBatch(TEST_DATA_SIZE);
    return [
      d.xs.reshape([TEST_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  const earlyStopping = tf.callbacks.earlyStopping({
    monitor: 'val_loss',
    patience: 3,  // stop if val_loss doesn't improve for 3 epochs
    restoreBestWeights: true,
  });

  return model.fit(trainXs, trainYs, {
    batchSize: BATCH_SIZE,
    validationData: [testXs, testYs],
    epochs: 20,
    shuffle: true,
    callbacks: [
      tfvis.show.fitCallbacks(
        { name: 'Training Performance' },
        ['loss', 'val_loss', 'acc', 'val_acc'],
        { height: 200, callbacks: ['onEpochEnd'] }
      ),
      earlyStopping
    ]
  });
}
```

✅ **Changes I made:**

- Added `earlyStopping` callback using `tf.callbacks.earlyStopping()`.
- Monitored `'val_loss'`.
- Set `patience: 3` (you can tweak it — it means *after 3 bad epochs, stop*).
- Also set `restoreBestWeights: true` so it *rolls back* to the best model automatically.

Would you also like me to show you **how to display a message** when early stopping kicks in? 🎯 (like: "**Training stopped early!**")
It's just a tiny extra!

yes i want console when earlystopng

Perfect! 🚀 Let's make it print a message when **early stopping happens**.

Here's the slightly **improved version** of your `trainModel` function:

```javascript

async function trainModel(model, data) {
  const BATCH_SIZE = 64;
  const TRAIN_DATA_SIZE = NUM_TRAIN_ELEMENTS;
  const TEST_DATA_SIZE = NUM_TEST_ELEMENTS;

  const [trainXs, trainYs] = tf.tidy(() => {
    const d = data.nextTrainBatch(TRAIN_DATA_SIZE);
    return [
      d.xs.reshape([TRAIN_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  const [testXs, testYs] = tf.tidy(() => {
    const d = data.nextTestBatch(TEST_DATA_SIZE);
    return [
      d.xs.reshape([TEST_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  const earlyStopping = tf.callbacks.earlyStopping({
    monitor: 'val_loss',
    patience: 3,
    restoreBestWeights: true,
    verbose: 1, // <-- add this (1 = show logs)
  });

  // Custom callback to detect early stopping
  const customCallback = {
    onTrainEnd: (logs) => {
      console.log('🛑 Training stopped. Early stopping triggered or epochs finished.');
    }
  };

  return model.fit(trainXs, trainYs, {
    batchSize: BATCH_SIZE,
    validationData: [testXs, testYs],
    epochs: 20,
    shuffle: true,
    callbacks: [
      tfvis.show.fitCallbacks(
        { name: 'Training Performance' },
        ['loss', 'val_loss', 'acc', 'val_acc'],
        { height: 200, callbacks: ['onEpochEnd'] }
      ),
      earlyStopping,
      customCallback
    ]
  });
}
```

✅ Now it will:

- **Show EarlyStopping info** inside the console because of `verbose: 1`.
- **When training ends** (either normally or because of early stopping), it will **print** this line:

```
🛑 Training stopped. Early stopping triggered or epochs finished.
```

Would you also like a **fancier** version where it tells you **after how many epochs** early stopping happened?
(even cleaner, like: "`Stopped after 7 epochs!`") 🎯🎯
I can add it if you want!

Uncaught (in promise) Error: restoreBestWeights = True is not implemented in EarlyStopping yet.
y https://cdn.jsdelivr.net/npm/@tensorflow/tfjs:17
r https://cdn.jsdelivr.net/npm/@tensorflow/tfjs:17
w https://cdn.jsdelivr.net/npm/@tensorflow/tfjs:17
n https://cdn.jsdelivr.net/npm/@tensorflow/tfjs:17
i https://cdn.jsdelivr.net/npm/@tensorflow/tfjs:17
earlyStopping https://cdn.jsdelivr.net/npm/@tensorflow/tfjs:17
trainModel http://localhost:8000/script.js:271
run http://localhost:8000/script.js:345
async* http://localhost:8000/script.js:350 const IMAGE_SIZE = 784;
const NUM_CLASSES = 10;
const NUM_DATASET_ELEMENTS = 65000;

const TRAIN_TEST_RATIO = 5 / 6;

const NUM_TRAIN_ELEMENTS = Math.floor(TRAIN_TEST_RATIO * NUM_DATASET_ELEMENTS);
const NUM_TEST_ELEMENTS = NUM_DATASET_ELEMENTS - NUM_TRAIN_ELEMENTS;

const MNIST_IMAGES_SPRITE_PATH =
'https://storage.googleapis.com/learnjs-data/model-builder/mnist_images.png';
const MNIST_LABELS_PATH =
'https://storage.googleapis.com/learnjs-data/model-builder/mnist_labels_uint8';

/**
* A class that fetches the sprited MNIST dataset and returns shuffled batches.
*
* NOTE: This will get much easier. For now, we do data fetching and
* manipulation manually.
*/
export class MnistData {
constructor() {
this.shuffledTrainIndex = 0;
this.shuffledTestIndex = 0;

```
}

async load() {
  // Make a request for the MNIST sprited image.
  const img = new Image();
  const canvas = document.createElement('canvas');
  const ctx = canvas.getContext('2d');
  const imgRequest = new Promise((resolve, reject) => {
    img.crossOrigin = '';
    img.onload = () => {
      img.width = img.naturalWidth;
      img.height = img.naturalHeight;

      const datasetBytesBuffer =
          new ArrayBuffer(NUM_DATASET_ELEMENTS * IMAGE_SIZE * 4);

      const chunkSize = 5000;
      canvas.width = img.width;
      canvas.height = chunkSize;

      for (let i = 0; i < NUM_DATASET_ELEMENTS / chunkSize; i++) {
        const datasetBytesView = new Float32Array(
            datasetBytesBuffer, i * IMAGE_SIZE * chunkSize * 4,
            IMAGE_SIZE * chunkSize);
        ctx.drawImage(
            img, 0, i * chunkSize, img.width, chunkSize, 0, 0, img.width,
            chunkSize);

        const imageData = ctx.getImageData(0, 0, canvas.width, canvas.height);

        for (let j = 0; j < imageData.data.length / 4; j++) {
          // All channels hold an equal value since the image is grayscale, so
          // just read the red channel.
          datasetBytesView[j] = imageData.data[j * 4] / 255;
        }
      }
      this.datasetImages = new Float32Array(datasetBytesBuffer);

      resolve();
    };
    img.src = MNIST_IMAGES_SPRITE_PATH;
  });

  const labelsRequest = fetch(MNIST_LABELS_PATH);
  const [imgResponse, labelsResponse] =
      await Promise.all([imgRequest, labelsRequest]);

  this.datasetLabels = new Uint8Array(await labelsResponse.arrayBuffer());

  // Create shuffled indices into the train/test set for when we select a
  // random dataset element for training / validation.
```

```
  this.trainIndices = tf.util.createShuffledIndices(NUM_TRAIN_ELEMENTS);
  this.testIndices = tf.util.createShuffledIndices(NUM_TEST_ELEMENTS);

  // Slice the the images and labels into train and test sets.
  this.trainImages =
      this.datasetImages.slice(0, IMAGE_SIZE * NUM_TRAIN_ELEMENTS);
  this.testImages = this.datasetImages.slice(IMAGE_SIZE * NUM_TRAIN_ELEMENTS);
  this.trainLabels =
      this.datasetLabels.slice(0, NUM_CLASSES * NUM_TRAIN_ELEMENTS);
  this.testLabels =
      this.datasetLabels.slice(NUM_CLASSES * NUM_TRAIN_ELEMENTS);
}

nextTrainBatch(batchSize) {
  return this.nextBatch(
      batchSize, [this.trainImages, this.trainLabels], () => {
        this.shuffledTrainIndex =
            (this.shuffledTrainIndex + 1) % this.trainIndices.length;
        return this.trainIndices[this.shuffledTrainIndex];
      });
}

nextTestBatch(batchSize) {
  return this.nextBatch(batchSize, [this.testImages, this.testLabels], () => {
    this.shuffledTestIndex =
        (this.shuffledTestIndex + 1) % this.testIndices.length;
    return this.testIndices[this.shuffledTestIndex];
  });
}

nextBatch(batchSize, data, index) {
  const batchImagesArray = new Float32Array(batchSize * IMAGE_SIZE);
  const batchLabelsArray = new Uint8Array(batchSize * NUM_CLASSES);

  for (let i = 0; i < batchSize; i++) {
    const idx = index();

    const image =
        data[0].slice(idx * IMAGE_SIZE, idx * IMAGE_SIZE + IMAGE_SIZE);
    batchImagesArray.set(image, i * IMAGE_SIZE);

    const label =
        data[1].slice(idx * NUM_CLASSES, idx * NUM_CLASSES + NUM_CLASSES);
    batchLabelsArray.set(label, i * NUM_CLASSES);
  }

  const xs = tf.tensor2d(batchImagesArray, [batchSize, IMAGE_SIZE]);
  const labels = tf.tensor2d(batchLabelsArray, [batchSize, NUM_CLASSES]);

  return {xs, labels};
}
```

```
}
let model;
let data;

// Drawing
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');
let isDrawing = false;

// Setup canvas correctly for MNIST
canvas.width = 280;
canvas.height = 280;
canvas.style.width = '280px';
canvas.style.height = '280px';
canvas.style.imageRendering = 'pixelated';

ctx.fillStyle = 'black'; // 🛠️ background BLACK like MNIST
ctx.fillRect(0, 0, canvas.width, canvas.height);

canvas.addEventListener('mousedown', () => isDrawing = true);
canvas.addEventListener('mouseup', () => {
  isDrawing = false;
  ctx.beginPath();
});
canvas.addEventListener('mouseout', () => {
  isDrawing = false;
  ctx.beginPath();
});
canvas.addEventListener('mousemove', draw);

function draw(e) {
  if (!isDrawing) return;
  ctx.lineWidth = 20; // 🛠️ thicker lines, like MNIST digits
  ctx.lineCap = 'round';
  ctx.strokeStyle = 'white'; // 🛠️ draw white on black

  const rect = canvas.getBoundingClientRect();
  const scaleX = canvas.width / rect.width;
  const scaleY = canvas.height / rect.height;

  ctx.lineTo((e.clientX - rect.left) * scaleX, (e.clientY - rect.top) * scaleY);
  ctx.stroke();
  ctx.beginPath();
  ctx.moveTo((e.clientX - rect.left) * scaleX, (e.clientY - rect.top) * scaleY);
}

document.getElementById('clearBtn').addEventListener('click', clearCanvas);
document.getElementById('predictBtn').addEventListener('click', predictCanvas);

function clearCanvas() {
  ctx.fillStyle = 'black'; // 🛠️ black background
```

```
  ctx.fillRect(0, 0, canvas.width, canvas.height);
  ctx.beginPath();
}

// Model
async function getModel() {
  const model = tf.sequential();
  const IMAGE_WIDTH = 28;
  const IMAGE_HEIGHT = 28;
  const IMAGE_CHANNELS = 1;

  model.add(tf.layers.conv2d({
    inputShape: [IMAGE_WIDTH, IMAGE_HEIGHT, IMAGE_CHANNELS],
    kernelSize: 5,
    filters: 8,
    strides: 1,
    activation: 'relu',
    kernelInitializer: 'varianceScaling'
  }));

  model.add(tf.layers.maxPooling2d({ poolSize: [2, 2], strides: [2, 2] }));

  model.add(tf.layers.conv2d({
    kernelSize: 5,
    filters: 16,
    strides: 1,
    activation: 'relu',
    kernelInitializer: 'varianceScaling'
  }));

  model.add(tf.layers.maxPooling2d({ poolSize: [2, 2], strides: [2, 2] }));

  model.add(tf.layers.flatten());

  model.add(tf.layers.dense({
    units: 10,
    kernelInitializer: 'varianceScaling',
    activation: 'softmax'
  }));

  const optimizer = tf.train.adam();
  model.compile({
    optimizer: optimizer,
    loss: 'categoricalCrossentropy',
    metrics: ['accuracy'],
  });

  return model;
}

async function trainModel(model, data) {
```

```
  const BATCH_SIZE = 64;
  const TRAIN_DATA_SIZE = NUM_TRAIN_ELEMENTS;
  const TEST_DATA_SIZE = NUM_TEST_ELEMENTS;

  const [trainXs, trainYs] = tf.tidy(() => {
    const d = data.nextTrainBatch(TRAIN_DATA_SIZE);
    return [
      d.xs.reshape([TRAIN_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  const [testXs, testYs] = tf.tidy(() => {
    const d = data.nextTestBatch(TEST_DATA_SIZE);
    return [
      d.xs.reshape([TEST_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  const earlyStopping = tf.callbacks.earlyStopping({
    monitor: 'val_loss',
    patience: 3,
    restoreBestWeights: true,
    verbose: 1, // <-- add this (1 = show logs)
  });

  // Custom callback to detect early stopping
  const customCallback = {
    onTrainEnd: (logs) => {
      console.log('🛑 Training stopped. Early stopping triggered or epochs finished.');
    }
  };

  return model.fit(trainXs, trainYs, {
    batchSize: BATCH_SIZE,
    validationData: [testXs, testYs],
    epochs: 20,
    shuffle: true,
    callbacks: [
      tfvis.show.fitCallbacks(
        { name: 'Training Performance' },
        ['loss', 'val_loss', 'acc', 'val_acc'],
        { height: 200, callbacks: ['onEpochEnd'] }
      ),
      earlyStopping,
      customCallback
    ]
  });
}
```

```
async function predictCanvas() {
  // Shrink big canvas down to 28x28
  const smallCanvas = document.createElement('canvas');
  smallCanvas.width = 28;
  smallCanvas.height = 28;
  const smallCtx = smallCanvas.getContext('2d');

  smallCtx.drawImage(canvas, 0, 0, 28, 28);

  const imgData = smallCtx.getImageData(0, 0, 28, 28);
  const data = imgData.data;

  const grayData = new Float32Array(28 * 28);

  for (let i = 0; i < data.length; i += 4) {
    // Remember: after draw, white pixel means drawn, black means background
    const r = data[i];
    const g = data[i + 1];
    const b = data[i + 2];
    const avg = (r + g + b) / 3;
    // Normalize to [0,1], background = 0, digit = 1
    grayData[i / 4] = avg / 255;
  }

  const input = tf.tensor4d(grayData, [1, 28, 28, 1]);

  const prediction = model.predict(input);
  const pred = prediction.argMax(1);
  const result = (await pred.data())[0];

  document.getElementById('prediction').innerText = `Prediction: ${result}`;

  // Dispose tensors
  input.dispose();
  prediction.dispose();
  pred.dispose();
}

// Run everything
async function run() {
  data = new MnistData();
  await data.load();
  model = await getModel();
  await trainModel(model, data);

  clearCanvas();
}

document.addEventListener('DOMContentLoaded', run);
```

Got it — the error you're seeing:

```
vbnet

Uncaught (in promise) Error: restoreBestWeights = True is not implemented in EarlyStopping yet.
```

happens because in **TensorFlow.js**, the `EarlyStopping` callback **does not yet support** the `restoreBestWeights: true` option (unlike TensorFlow Python).
👉 That means `restoreBestWeights: true` **is not available in tfjs yet** — so it crashes.

Quick fix:

You **must remove** `restoreBestWeights: true` from your `earlyStopping` callback.

Change this part in your `trainModel`:

```
javascript

const earlyStopping = tf.callbacks.earlyStopping({
  monitor: 'val_loss',
  patience: 3,
  verbose: 1, // keep this!
});
```

✅ Just **delete** `restoreBestWeights: true`.

And about your original goal:

You said:

> yes i want console when early stopping.

Don't worry — your `customCallback` with `onTrainEnd` **already does that!**
When early stopping happens or all epochs finish, you will see in console:

```
🛑 Training stopped. Early stopping triggered or epochs finished.
```

Nothing else needed.
You only had to remove `restoreBestWeights: true`.

Full working `trainModel` snippet:

```
javascript

async function trainModel(model, data) {
  const BATCH_SIZE = 64;
  const TRAIN_DATA_SIZE = NUM_TRAIN_ELEMENTS;
  const TEST_DATA_SIZE = NUM_TEST_ELEMENTS;

  const [trainXs, trainYs] = tf.tidy(() => {
    const d = data.nextTrainBatch(TRAIN_DATA_SIZE);
    return [
      d.xs.reshape([TRAIN_DATA_SIZE, 28, 28, 1]),
      d.labels
```

```
    ];
  });

  const [testXs, testYs] = tf.tidy(() => {
    const d = data.nextTestBatch(TEST_DATA_SIZE);
    return [
      d.xs.reshape([TEST_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  const earlyStopping = tf.callbacks.earlyStopping({
    monitor: 'val_loss',
    patience: 3,
    verbose: 1,
  });

  const customCallback = {
    onTrainEnd: (logs) => {
      console.log('🛑 Training stopped. Early stopping triggered or epochs finished.');
    }
  };

  return model.fit(trainXs, trainYs, {
    batchSize: BATCH_SIZE,
    validationData: [testXs, testYs],
    epochs: 20,
    shuffle: true,
    callbacks: [
      tfvis.show.fitCallbacks(
        { name: 'Training Performance' },
        ['loss', 'val_loss', 'acc', 'val_acc'],
        { height: 200, callbacks: ['onEpochEnd'] }
      ),
      earlyStopping,
      customCallback
    ]
  });
}
```

Would you also like me to show you how to **manually restore best weights** if you really want that feature?
(It's a little workaround since `restoreBestWeights` is not supported yet.) 🚀

Uncaught (in promise) TypeError: this.getMonitorValue is not a function

Got it — let's break this down carefully.

The **first error** you had earlier:

```
vbnet

Uncaught (in promise) Error: restoreBestWeights = True is not implemented in EarlyStopping yet.
```

tells us that **TensorFlow.js** (the version you're using) **does not support** `restoreBestWeights: true` for `tf.callbacks.earlyStopping()`.

The **new error** you're seeing now:

```
javascript

Uncaught (in promise) TypeError: this.getMonitorValue is not a function
```

is happening because after the first crash, the callback is still being created with an unsupported feature. Internally, TensorFlow.js tries to access a method that doesn't exist because of the wrong callback setup.

✅ **Solution:** You **cannot** use `restoreBestWeights: true` in TensorFlow.js yet (as of the version you loaded from the CDN).
You need to **remove** `restoreBestWeights: true` from your `earlyStopping` callback completely.

Here's the **fixed** part of your `trainModel` function:

```
javascript

const earlyStopping = tf.callbacks.earlyStopping({
  monitor: 'val_loss',
  patience: 3,
  verbose: 1, // show logs
  // ❌ remove restoreBestWeights: true
});
```

Why?

- In TensorFlow **Python**, `restore_best_weights` is supported.
- In TensorFlow.js, **it's not implemented yet**.
- If you include it, TensorFlow.js throws unexpected internal errors like `getMonitorValue is not a function`.

Quick Summary:

- **Delete** `restoreBestWeights: true` from `earlyStopping`
- Keep `verbose: 1` if you want training logs.

Would you like me to also show you a **simple way to manually save and restore best weights** without using `restoreBestWeights`, if you still want that behavior? 🎯
(It's a small trick we can add manually.)

i dont have this line. const IMAGE_SIZE = 784;
const NUM_CLASSES = 10;
const NUM_DATASET_ELEMENTS = 65000;

const TRAIN_TEST_RATIO = 5 / 6;

const NUM_TRAIN_ELEMENTS = Math.floor(TRAIN_TEST_RATIO * NUM_DATASET_ELEMENTS);
const NUM_TEST_ELEMENTS = NUM_DATASET_ELEMENTS - NUM_TRAIN_ELEMENTS;

```
const MNIST_IMAGES_SPRITE_PATH =
    'https://storage.googleapis.com/learnjs-data/model-builder/mnist_images.png';
const MNIST_LABELS_PATH =
    'https://storage.googleapis.com/learnjs-data/model-builder/mnist_labels_uint8';

/**
 * A class that fetches the sprited MNIST dataset and returns shuffled batches.
 *
 * NOTE: This will get much easier. For now, we do data fetching and
 * manipulation manually.
 */
export class MnistData {
  constructor() {
    this.shuffledTrainIndex = 0;
    this.shuffledTestIndex = 0;
  }

  async load() {
    // Make a request for the MNIST sprited image.
    const img = new Image();
    const canvas = document.createElement('canvas');
    const ctx = canvas.getContext('2d');
    const imgRequest = new Promise((resolve, reject) => {
      img.crossOrigin = '';
      img.onload = () => {
        img.width = img.naturalWidth;
        img.height = img.naturalHeight;

        const datasetBytesBuffer =
            new ArrayBuffer(NUM_DATASET_ELEMENTS * IMAGE_SIZE * 4);

        const chunkSize = 5000;
        canvas.width = img.width;
        canvas.height = chunkSize;

        for (let i = 0; i < NUM_DATASET_ELEMENTS / chunkSize; i++) {
          const datasetBytesView = new Float32Array(
              datasetBytesBuffer, i * IMAGE_SIZE * chunkSize * 4,
              IMAGE_SIZE * chunkSize);
          ctx.drawImage(
              img, 0, i * chunkSize, img.width, chunkSize, 0, 0, img.width,
              chunkSize);

          const imageData = ctx.getImageData(0, 0, canvas.width, canvas.height);

          for (let j = 0; j < imageData.data.length / 4; j++) {
            // All channels hold an equal value since the image is grayscale, so
            // just read the red channel.
            datasetBytesView[j] = imageData.data[j * 4] / 255;
          }
```

```
      }
      this.datasetImages = new Float32Array(datasetBytesBuffer);

      resolve();
    };
    img.src = MNIST_IMAGES_SPRITE_PATH;
  });

  const labelsRequest = fetch(MNIST_LABELS_PATH);
  const [imgResponse, labelsResponse] =
      await Promise.all([imgRequest, labelsRequest]);

  this.datasetLabels = new Uint8Array(await labelsResponse.arrayBuffer());

  // Create shuffled indices into the train/test set for when we select a
  // random dataset element for training / validation.
  this.trainIndices = tf.util.createShuffledIndices(NUM_TRAIN_ELEMENTS);
  this.testIndices = tf.util.createShuffledIndices(NUM_TEST_ELEMENTS);

  // Slice the the images and labels into train and test sets.
  this.trainImages =
      this.datasetImages.slice(0, IMAGE_SIZE * NUM_TRAIN_ELEMENTS);
  this.testImages = this.datasetImages.slice(IMAGE_SIZE * NUM_TRAIN_ELEMENTS);
  this.trainLabels =
      this.datasetLabels.slice(0, NUM_CLASSES * NUM_TRAIN_ELEMENTS);
  this.testLabels =
      this.datasetLabels.slice(NUM_CLASSES * NUM_TRAIN_ELEMENTS);
}

nextTrainBatch(batchSize) {
  return this.nextBatch(
      batchSize, [this.trainImages, this.trainLabels], () => {
        this.shuffledTrainIndex =
            (this.shuffledTrainIndex + 1) % this.trainIndices.length;
        return this.trainIndices[this.shuffledTrainIndex];
      });
}

nextTestBatch(batchSize) {
  return this.nextBatch(batchSize, [this.testImages, this.testLabels], () => {
    this.shuffledTestIndex =
        (this.shuffledTestIndex + 1) % this.testIndices.length;
    return this.testIndices[this.shuffledTestIndex];
  });
}

nextBatch(batchSize, data, index) {
  const batchImagesArray = new Float32Array(batchSize * IMAGE_SIZE);
  const batchLabelsArray = new Uint8Array(batchSize * NUM_CLASSES);

  for (let i = 0; i < batchSize; i++) {
```

```
      const idx = index();

      const image =
          data[0].slice(idx * IMAGE_SIZE, idx * IMAGE_SIZE + IMAGE_SIZE);
      batchImagesArray.set(image, i * IMAGE_SIZE);

      const label =
          data[1].slice(idx * NUM_CLASSES, idx * NUM_CLASSES + NUM_CLASSES);
      batchLabelsArray.set(label, i * NUM_CLASSES);
    }

    const xs = tf.tensor2d(batchImagesArray, [batchSize, IMAGE_SIZE]);
    const labels = tf.tensor2d(batchLabelsArray, [batchSize, NUM_CLASSES]);

    return {xs, labels};
  }
}
let model;
let data;

// Drawing
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');
let isDrawing = false;

// Setup canvas correctly for MNIST
canvas.width = 280;
canvas.height = 280;
canvas.style.width = '280px';
canvas.style.height = '280px';
canvas.style.imageRendering = 'pixelated';

ctx.fillStyle = 'black'; // 🛠️ background BLACK like MNIST
ctx.fillRect(0, 0, canvas.width, canvas.height);

canvas.addEventListener('mousedown', () => isDrawing = true);
canvas.addEventListener('mouseup', () => {
  isDrawing = false;
  ctx.beginPath();
});
canvas.addEventListener('mouseout', () => {
  isDrawing = false;
  ctx.beginPath();
});
canvas.addEventListener('mousemove', draw);

function draw(e) {
  if (!isDrawing) return;
  ctx.lineWidth = 20; // 🛠️ thicker lines, like MNIST digits
  ctx.lineCap = 'round';
  ctx.strokeStyle = 'white'; // 🛠️ draw white on black
```

```
  const rect = canvas.getBoundingClientRect();
  const scaleX = canvas.width / rect.width;
  const scaleY = canvas.height / rect.height;

  ctx.lineTo((e.clientX - rect.left) * scaleX, (e.clientY - rect.top) * scaleY);
  ctx.stroke();
  ctx.beginPath();
  ctx.moveTo((e.clientX - rect.left) * scaleX, (e.clientY - rect.top) * scaleY);
}

document.getElementById('clearBtn').addEventListener('click', clearCanvas);
document.getElementById('predictBtn').addEventListener('click', predictCanvas);

function clearCanvas() {
  ctx.fillStyle = 'black'; // 🛠️ black background
  ctx.fillRect(0, 0, canvas.width, canvas.height);
  ctx.beginPath();
}

// Model
async function getModel() {
  const model = tf.sequential();
  const IMAGE_WIDTH = 28;
  const IMAGE_HEIGHT = 28;
  const IMAGE_CHANNELS = 1;

  model.add(tf.layers.conv2d({
    inputShape: [IMAGE_WIDTH, IMAGE_HEIGHT, IMAGE_CHANNELS],
    kernelSize: 5,
    filters: 8,
    strides: 1,
    activation: 'relu',
    kernelInitializer: 'varianceScaling'
  }));

  model.add(tf.layers.maxPooling2d({ poolSize: [2, 2], strides: [2, 2] }));

  model.add(tf.layers.conv2d({
    kernelSize: 5,
    filters: 16,
    strides: 1,
    activation: 'relu',
    kernelInitializer: 'varianceScaling'
  }));

  model.add(tf.layers.maxPooling2d({ poolSize: [2, 2], strides: [2, 2] }));

  model.add(tf.layers.flatten());

  model.add(tf.layers.dense({
```

```
    units: 10,
    kernelInitializer: 'varianceScaling',
    activation: 'softmax'
  }));

  const optimizer = tf.train.adam();
  model.compile({
    optimizer: optimizer,
    loss: 'categoricalCrossentropy',
    metrics: ['accuracy'],
  });

  return model;
}

async function trainModel(model, data) {
  const BATCH_SIZE = 64;
  const TRAIN_DATA_SIZE = NUM_TRAIN_ELEMENTS;
  const TEST_DATA_SIZE = NUM_TEST_ELEMENTS;

  const [trainXs, trainYs] = tf.tidy(() => {
    const d = data.nextTrainBatch(TRAIN_DATA_SIZE);
    return [
      d.xs.reshape([TRAIN_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  const [testXs, testYs] = tf.tidy(() => {
    const d = data.nextTestBatch(TEST_DATA_SIZE);
    return [
      d.xs.reshape([TEST_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  const earlyStopping = tf.callbacks.earlyStopping({
    monitor: 'val_loss',
    patience: 3,
    verbose: 1,
  });

  const customCallback = {
    onTrainEnd: (logs) => {
      console.log('Training stopped. Early stopping triggered or epochs finished.');
    }
  };

  return model.fit(trainXs, trainYs, {
    batchSize: BATCH_SIZE,
    validationData: [testXs, testYs],
```

```
    epochs: 20,
    shuffle: true,
    callbacks: [
      tfvis.show.fitCallbacks(
        { name: 'Training Performance' },
        ['loss', 'val_loss', 'acc', 'val_acc'],
        { height: 200, callbacks: ['onEpochEnd'] }
      ),
      earlyStopping,
      customCallback
    ]
  });
}


async function predictCanvas() {
  // Shrink big canvas down to 28x28
  const smallCanvas = document.createElement('canvas');
  smallCanvas.width = 28;
  smallCanvas.height = 28;
  const smallCtx = smallCanvas.getContext('2d');

  smallCtx.drawImage(canvas, 0, 0, 28, 28);

  const imgData = smallCtx.getImageData(0, 0, 28, 28);
  const data = imgData.data;

  const grayData = new Float32Array(28 * 28);

  for (let i = 0; i < data.length; i += 4) {
    // Remember: after draw, white pixel means drawn, black means background
    const r = data[i];
    const g = data[i + 1];
    const b = data[i + 2];
    const avg = (r + g + b) / 3;
    // Normalize to [0,1], background = 0, digit = 1
    grayData[i / 4] = avg / 255;
  }

  const input = tf.tensor4d(grayData, [1, 28, 28, 1]);

  const prediction = model.predict(input);
  const pred = prediction.argMax(1);
  const result = (await pred.data())[0];

  document.getElementById('prediction').innerText = `Prediction: ${result}`;

  // Dispose tensors
  input.dispose();
  prediction.dispose();
  pred.dispose();
```

```
}

// Run everything
async function run() {
  data = new MnistData();
  await data.load();
  model = await getModel();
  await trainModel(model, data);

  clearCanvas();
}

document.addEventListener('DOMContentLoaded', run);
```

Okay, I see your code — very clean and detailed!
The error you're getting:

```
javascript

Uncaught (in promise) TypeError: this.getMonitorValue is not a function
```

happens because inside your `**trainModel()**` function you have this part:

```
javascript

tfvis.show.fitCallbacks(
  { name: 'Training Performance' },
  ['loss', 'val_loss', 'acc', 'val_acc'],
  { height: 200, callbacks: ['onEpochEnd'] }
)
```

but `**acc**` and `**val_acc**` are **wrong metric names** for TensorFlow.js now!

In TensorFlow.js **the metric is called** `**'accuracy'**`, not `**'acc'**`.
Same for validation: `**'val_accuracy'**`, not `**'val_acc'**`.

🔵 Correct list:

```
javascript

tfvis.show.fitCallbacks(
  { name: 'Training Performance' },
  ['loss', 'val_loss', 'accuracy', 'val_accuracy'],
  { height: 200, callbacks: ['onEpochEnd'] }
)
```

✅ **Summary of what you need to change:**
Find:

```
javascript

['loss', 'val_loss', 'acc', 'val_acc']
```

replace with:

```
javascript

['loss', 'val_loss', 'accuracy', 'val_accuracy']
```

Would you like me to also show you a little improvement so your `predictCanvas()` result text doesn't get cut off? (just a small touch!) 🚀

how to export const NUM_TRAIN_ELEMENTS and NUM_TEST_ELEMENTS so i can use in script.js

Got it — you want to **use `NUM_TRAIN_ELEMENTS` and `NUM_TEST_ELEMENTS` inside `script.js`**, but currently they're defined inside another module, right?

✅ Here's exactly how you can do it:

First, in the file where you define them (it looks like your `mnistData.js` or similar), **export** them:

```
javascript

// mnistData.js

export const NUM_TRAIN_ELEMENTS = Math.floor(TRAIN_TEST_RATIO * NUM_DATASET_ELEMENTS);
export const NUM_TEST_ELEMENTS = NUM_DATASET_ELEMENTS - NUM_TRAIN_ELEMENTS;
```

(you already have them calculated — just add `export` before them)

Then, **in your `script.js`**, you can **import** them like this:

```
javascript

import { NUM_TRAIN_ELEMENTS, NUM_TEST_ELEMENTS } from './mnistData.js';
```

⚡ **Important:**
Make sure the path `./mnistData.js` matches **exactly** where your file is relative to `script.js`.
For example, if it's in the same folder, `./mnistData.js` is fine.
If it's inside a folder like `/js/mnistData.js`, you would import from `'./js/mnistData.js'`.

One more thing:
Your `<script>` tag in HTML must have `type="module"` to allow imports/exports:

```
html

<script type="module" src="script.js"></script>
```

Otherwise you'll get errors like "unexpected token export"!

Summary in 3 steps:

1. Add `export` before `NUM_TRAIN_ELEMENTS` and `NUM_TEST_ELEMENTS`.
2. `import { NUM_TRAIN_ELEMENTS, NUM_TEST_ELEMENTS } from './your-file.js';`

3. Make sure your `<script>` has `type="module"`.

Would you like me to also show you a clean version of how to refactor your `mnistData.js` if you want it even nicer? 🚀
(For example, grouping constants and exporting them at once.)

i got loss 1.2 and accuracy 0.55. how to make this parameters more accurate. // Model
async function getModel() {
  const model = tf.sequential();
  const IMAGE_WIDTH = 28;
  const IMAGE_HEIGHT = 28;
  const IMAGE_CHANNELS = 1;

  model.add(tf.layers.conv2d({
    inputShape: [IMAGE_WIDTH, IMAGE_HEIGHT, IMAGE_CHANNELS],
    kernelSize: 5,
    filters: 8,
    strides: 1,
    activation: 'relu',
    kernelInitializer: 'varianceScaling'
  }));

  model.add(tf.layers.maxPooling2d({ poolSize: [2, 2], strides: [2, 2] }));

  model.add(tf.layers.conv2d({
    kernelSize: 5,
    filters: 16,
    strides: 1,
    activation: 'relu',
    kernelInitializer: 'varianceScaling'
  }));

  model.add(tf.layers.maxPooling2d({ poolSize: [2, 2], strides: [2, 2] }));

  model.add(tf.layers.flatten());

  model.add(tf.layers.dense({
    units: 10,
    kernelInitializer: 'varianceScaling',
    activation: 'softmax'
  }));

  const optimizer = tf.train.adam();
  model.compile({
    optimizer: optimizer,
    loss: 'categoricalCrossentropy',
    metrics: ['accuracy'],
  });

```
  return model;
}

async function trainModel(model, data) {
  const BATCH_SIZE = 512;
  const TRAIN_DATA_SIZE = NUM_TRAIN_ELEMENTS ;
  const TEST_DATA_SIZE = NUM_TEST_ELEMENTS;

  const [trainXs, trainYs] = tf.tidy(() => {
    const d = data.nextTrainBatch(TRAIN_DATA_SIZE);
    return [
      d.xs.reshape([TRAIN_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  const [testXs, testYs] = tf.tidy(() => {
    const d = data.nextTestBatch(TEST_DATA_SIZE);
    return [
      d.xs.reshape([TEST_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  return model.fit(trainXs, trainYs, {
    batchSize: BATCH_SIZE,
    validationData: [testXs, testYs],
    epochs: 100,
    shuffle: true,
    callbacks: tfvis.show.fitCallbacks(
      { name: 'Training Performance' },
      ['loss', 'val_loss', 'acc', 'val_acc'],
      { height: 200, callbacks: ['onEpochEnd'] }
    )
  });
}

async function predictCanvas() {
  // Shrink big canvas down to 28x28
  const smallCanvas = document.createElement('canvas');
  smallCanvas.width = 28;
  smallCanvas.height = 28;
  const smallCtx = smallCanvas.getContext('2d');

  smallCtx.drawImage(canvas, 0, 0, 28, 28);

  const imgData = smallCtx.getImageData(0, 0, 28, 28);
  const data = imgData.data;

  const grayData = new Float32Array(28 * 28);
```

```
  for (let i = 0; i < data.length; i += 4) {
    // Remember: after draw, white pixel means drawn, black means background
    const r = data[i];
    const g = data[i + 1];
    const b = data[i + 2];
    const avg = (r + g + b) / 3;
    // Normalize to [0,1], background = 0, digit = 1
    grayData[i / 4] = avg / 255;
  }

  const input = tf.tensor4d(grayData, [1, 28, 28, 1]);

  const prediction = model.predict(input);
  const pred = prediction.argMax(1);
  const result = (await pred.data())[0];

  document.getElementById('prediction').innerText = `Prediction: ${result}`;

  // Dispose tensors
  input.dispose();
  prediction.dispose();
  pred.dispose();
}

// Run everything
async function run() {
  data = new MnistData();
  await data.load();
  model = await getModel();
  await trainModel(model, data);

  clearCanvas();
}

document.addEventListener('DOMContentLoaded', run);  const IMAGE_SIZE = 784;
const NUM_CLASSES = 10;
const NUM_DATASET_ELEMENTS = 65000;

const TRAIN_TEST_RATIO = 5 / 6;

const NUM_TRAIN_ELEMENTS = Math.floor(TRAIN_TEST_RATIO * NUM_DATASET_ELEMENTS);
const NUM_TEST_ELEMENTS = NUM_DATASET_ELEMENTS - NUM_TRAIN_ELEMENTS;

const MNIST_IMAGES_SPRITE_PATH =
    'https://storage.googleapis.com/learnjs-data/model-builder/mnist_images.png';
const MNIST_LABELS_PATH =
    'https://storage.googleapis.com/learnjs-data/model-builder/mnist_labels_uint8';

/**
 * A class that fetches the sprited MNIST dataset and returns shuffled batches.
```

```
 *
 * NOTE: This will get much easier. For now, we do data fetching and
 * manipulation manually.
 */
export class MnistData {
  constructor() {
    this.shuffledTrainIndex = 0;
    this.shuffledTestIndex = 0;
  }

  async load() {
    // Make a request for the MNIST sprited image.
    const img = new Image();
    const canvas = document.createElement('canvas');
    const ctx = canvas.getContext('2d');
    const imgRequest = new Promise((resolve, reject) => {
      img.crossOrigin = '';
      img.onload = () => {
        img.width = img.naturalWidth;
        img.height = img.naturalHeight;

        const datasetBytesBuffer =
            new ArrayBuffer(NUM_DATASET_ELEMENTS * IMAGE_SIZE * 4);

        const chunkSize = 5000;
        canvas.width = img.width;
        canvas.height = chunkSize;

        for (let i = 0; i < NUM_DATASET_ELEMENTS / chunkSize; i++) {
          const datasetBytesView = new Float32Array(
              datasetBytesBuffer, i * IMAGE_SIZE * chunkSize * 4,
              IMAGE_SIZE * chunkSize);
          ctx.drawImage(
              img, 0, i * chunkSize, img.width, chunkSize, 0, 0, img.width,
              chunkSize);

          const imageData = ctx.getImageData(0, 0, canvas.width, canvas.height);

          for (let j = 0; j < imageData.data.length / 4; j++) {
            // All channels hold an equal value since the image is grayscale, so
            // just read the red channel.
            datasetBytesView[j] = imageData.data[j * 4] / 255;
          }
        }
        this.datasetImages = new Float32Array(datasetBytesBuffer);

        resolve();
      };
      img.src = MNIST_IMAGES_SPRITE_PATH;
    });
```

```
  const labelsRequest = fetch(MNIST_LABELS_PATH);
  const [imgResponse, labelsResponse] =
      await Promise.all([imgRequest, labelsRequest]);

  this.datasetLabels = new Uint8Array(await labelsResponse.arrayBuffer());

  // Create shuffled indices into the train/test set for when we select a
  // random dataset element for training / validation.
  this.trainIndices = tf.util.createShuffledIndices(NUM_TRAIN_ELEMENTS);
  this.testIndices = tf.util.createShuffledIndices(NUM_TEST_ELEMENTS);

  // Slice the the images and labels into train and test sets.
  this.trainImages =
      this.datasetImages.slice(0, IMAGE_SIZE * NUM_TRAIN_ELEMENTS);
  this.testImages = this.datasetImages.slice(IMAGE_SIZE * NUM_TRAIN_ELEMENTS);
  this.trainLabels =
      this.datasetLabels.slice(0, NUM_CLASSES * NUM_TRAIN_ELEMENTS);
  this.testLabels =
      this.datasetLabels.slice(NUM_CLASSES * NUM_TRAIN_ELEMENTS);
}

nextTrainBatch(batchSize) {
  return this.nextBatch(
      batchSize, [this.trainImages, this.trainLabels], () => {
        this.shuffledTrainIndex =
            (this.shuffledTrainIndex + 1) % this.trainIndices.length;
        return this.trainIndices[this.shuffledTrainIndex];
      });
}

nextTestBatch(batchSize) {
  return this.nextBatch(batchSize, [this.testImages, this.testLabels], () => {
    this.shuffledTestIndex =
        (this.shuffledTestIndex + 1) % this.testIndices.length;
    return this.testIndices[this.shuffledTestIndex];
  });
}

nextBatch(batchSize, data, index) {
  const batchImagesArray = new Float32Array(batchSize * IMAGE_SIZE);
  const batchLabelsArray = new Uint8Array(batchSize * NUM_CLASSES);

  for (let i = 0; i < batchSize; i++) {
    const idx = index();

    const image =
        data[0].slice(idx * IMAGE_SIZE, idx * IMAGE_SIZE + IMAGE_SIZE);
    batchImagesArray.set(image, i * IMAGE_SIZE);

    const label =
        data[1].slice(idx * NUM_CLASSES, idx * NUM_CLASSES + NUM_CLASSES);
```

```
      batchLabelsArray.set(label, i * NUM_CLASSES);
    }

    const xs = tf.tensor2d(batchImagesArray, [batchSize, IMAGE_SIZE]);
    const labels = tf.tensor2d(batchLabelsArray, [batchSize, NUM_CLASSES]);

    return {xs, labels};
  }
}
```

Good job setting everything up — your project is really close to being strong!

You said **loss ~1.2** and **accuracy ~0.55** (55%) —
✅ **that's decent for first CNN attempt on MNIST, but you can get 98%-99% accuracy.**

To **improve your model's accuracy**, here are **specific actions** you can take:

🔵 1. Add a Dense Hidden Layer

Right now, you **go straight from flatten() to softmax**. That's too simple!

➔ Add **one more dense layer** between `` `flatten()` `` and `` `dense({units: 10})` ``:

```
javascript

model.add(tf.layers.dense({
  units: 64,          // Try 64 or 128 neurons
  activation: 'relu',
  kernelInitializer: 'varianceScaling'
}));
```

Then your final softmax layer stays the same.

🔵 2. Increase Filters in Convolution Layers

Currently you use:

- 8 filters ➔ 16 filters

Better: **start bigger**, for example:

```
javascript

filters: 32   // instead of 8
filters: 64   // instead of 16
```

So **double** your filters at least. MNIST is simple, but 8 is too small.

🔵 3. Use a Smaller Batch Size

You are using `` `BATCH_SIZE = 512` ``, which is **too big**.

➔ Try **smaller batch sizes** like **128** or **256**.

Why?
Smaller batches = **better generalization** (model learns noise patterns better).

```
javascript

const BATCH_SIZE = 128;
```

🔵 4. Normalize Input More Carefully

You are normalizing your canvas manually to `[0,1]`, but in training your images are also normalized right?

✅ Your dataset from `MnistData` seems already normalized, so that's good.

But **make sure** you **reverse color** if needed — sometimes MNIST expects **white digit on black**, or **black digit on white**.
If needed you can invert:

```
javascript

grayData[i / 4] = 1 - avg / 255;
```

This **small detail** can **boost your accuracy** by a lot!

🔵 5. Train for Longer or with Early Stopping

You use `epochs: 100`, but are you watching overfitting?
If you overfit, the validation loss will go up.

Later you can:

- Add `earlyStopping` callbacks,
- Lower learning rate dynamically (advanced).

For now, **100 epochs** is OK.

🔵 6. Use Data Augmentation (Optional)

Later you can **augment your data**: random shifts, rotations, etc.

But for now, MNIST is simple enough — you don't *need* this yet.

✨ Here's a new model version for you:

```
javascript

async function getModel() {
  const model = tf.sequential();
  const IMAGE_WIDTH = 28;
  const IMAGE_HEIGHT = 28;
  const IMAGE_CHANNELS = 1;
```

```
  model.add(tf.layers.conv2d({
    inputShape: [IMAGE_WIDTH, IMAGE_HEIGHT, IMAGE_CHANNELS],
    kernelSize: 5,
    filters: 32,   // was 8
    strides: 1,
    activation: 'relu',
    kernelInitializer: 'varianceScaling'
  }));

  model.add(tf.layers.maxPooling2d({ poolSize: [2, 2], strides: [2, 2] }));

  model.add(tf.layers.conv2d({
    kernelSize: 5,
    filters: 64,   // was 16
    strides: 1,
    activation: 'relu',
    kernelInitializer: 'varianceScaling'
  }));

  model.add(tf.layers.maxPooling2d({ poolSize: [2, 2], strides: [2, 2] }));

  model.add(tf.layers.flatten());

  model.add(tf.layers.dense({   // new hidden dense layer
    units: 64,
    activation: 'relu',
    kernelInitializer: 'varianceScaling'
  }));

  model.add(tf.layers.dense({
    units: 10,
    activation: 'softmax',
    kernelInitializer: 'varianceScaling'
  }));

  const optimizer = tf.train.adam();
  model.compile({
    optimizer: optimizer,
    loss: 'categoricalCrossentropy',
    metrics: ['accuracy'],
  });

  return model;
}
```

And change batch size:

```
javascript

const BATCH_SIZE = 128;   // was 512
```

🔥 After these changes you should see:

Metric	Before	After
Accuracy	55%	**95-98%**
Loss	1.2	**0.05 - 0.2**

Would you also like me to show you **how to add early stopping** if validation loss increases? 🚀
(This way training automatically stops at best epoch.)
It would make your model even better!
Want me to?

is there way to save trained model, and load it if exists without new training (if not exists - train).

```
async function getModel() {
  const model = tf.sequential();
  const IMAGE_WIDTH = 28;
  const IMAGE_HEIGHT = 28;
  const IMAGE_CHANNELS = 1;

  model.add(tf.layers.conv2d({
    inputShape: [IMAGE_WIDTH, IMAGE_HEIGHT, IMAGE_CHANNELS],
    kernelSize: 5,
    filters: 32,   // was 8
    strides: 1,
    activation: 'relu',
    kernelInitializer: 'varianceScaling'
  }));

  model.add(tf.layers.maxPooling2d({ poolSize: [2, 2], strides: [2, 2] }));

  model.add(tf.layers.conv2d({
    kernelSize: 5,
    filters: 64,   // was 16
    strides: 1,
    activation: 'relu',
    kernelInitializer: 'varianceScaling'
  }));

  model.add(tf.layers.maxPooling2d({ poolSize: [2, 2], strides: [2, 2] }));

  model.add(tf.layers.flatten());

  model.add(tf.layers.dense({   // new hidden dense layer
    units: 64,
    activation: 'relu',
    kernelInitializer: 'varianceScaling'
  }));

  model.add(tf.layers.dense({
    units: 10,
    activation: 'softmax',
    kernelInitializer: 'varianceScaling'
  }));

  const optimizer = tf.train.adam();
  model.compile({
    optimizer: optimizer,
    loss: 'categoricalCrossentropy',
    metrics: ['accuracy'],
  });
```

```
  return model;
}


async function trainModel(model, data) {
  const BATCH_SIZE = 128;
  const TRAIN_DATA_SIZE = NUM_TRAIN_ELEMENTS ;
  const TEST_DATA_SIZE = NUM_TEST_ELEMENTS;

  const [trainXs, trainYs] = tf.tidy(() => {
    const d = data.nextTrainBatch(TRAIN_DATA_SIZE);
    return [
      d.xs.reshape([TRAIN_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  const [testXs, testYs] = tf.tidy(() => {
    const d = data.nextTestBatch(TEST_DATA_SIZE);
    return [
      d.xs.reshape([TEST_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  return model.fit(trainXs, trainYs, {
    batchSize: BATCH_SIZE,
    validationData: [testXs, testYs],
    epochs: 10,
    shuffle: true,
    callbacks: tfvis.show.fitCallbacks(
      { name: 'Training Performance' },
      ['loss', 'val_loss', 'acc', 'val_acc'],
      { height: 200, callbacks: ['onEpochEnd'] }
    )
  });
}

async function predictCanvas() {
  // Shrink big canvas down to 28x28
  const smallCanvas = document.createElement('canvas');
  smallCanvas.width = 28;
  smallCanvas.height = 28;
  const smallCtx = smallCanvas.getContext('2d');

  smallCtx.drawImage(canvas, 0, 0, 28, 28);

  const imgData = smallCtx.getImageData(0, 0, 28, 28);
  const data = imgData.data;
```

```
  const grayData = new Float32Array(28 * 28);

  for (let i = 0; i < data.length; i += 4) {
    // Remember: after draw, white pixel means drawn, black means background
    const r = data[i];
    const g = data[i + 1];
    const b = data[i + 2];
    const avg = (r + g + b) / 3;
    // Normalize to [0,1], background = 0, digit = 1
    grayData[i / 4] = avg / 255;
  }

  const input = tf.tensor4d(grayData, [1, 28, 28, 1]);

  const prediction = model.predict(input);
  const pred = prediction.argMax(1);
  const result = (await pred.data())[0];

  document.getElementById('prediction').innerText = `Prediction: ${result}`;

  // Dispose tensors
  input.dispose();
  prediction.dispose();
  pred.dispose();
}

// Run everything
async function run() {
  data = new MnistData();
  await data.load();
  model = await getModel();
  await trainModel(model, data);

  clearCanvas();
}

document.addEventListener('DOMContentLoaded', run);  const IMAGE_SIZE = 784;
const NUM_CLASSES = 10;
const NUM_DATASET_ELEMENTS = 65000;

const TRAIN_TEST_RATIO = 5 / 6;

export const NUM_TRAIN_ELEMENTS = Math.floor(TRAIN_TEST_RATIO * NUM_DATASET_ELEMENTS);
export const NUM_TEST_ELEMENTS = NUM_DATASET_ELEMENTS - NUM_TRAIN_ELEMENTS;

const MNIST_IMAGES_SPRITE_PATH =
    'https://storage.googleapis.com/learnjs-data/model-builder/mnist_images.png';
const MNIST_LABELS_PATH =
    'https://storage.googleapis.com/learnjs-data/model-builder/mnist_labels_uint8';

/**
```

```
 * A class that fetches the sprited MNIST dataset and returns shuffled batches.
 *
 * NOTE: This will get much easier. For now, we do data fetching and
 * manipulation manually.
 */
export class MnistData {
 constructor() {
  this.shuffledTrainIndex = 0;
  this.shuffledTestIndex = 0;
 }

 async load() {
  // Make a request for the MNIST sprited image.
  const img = new Image();
  const canvas = document.createElement('canvas');
  const ctx = canvas.getContext('2d');
  const imgRequest = new Promise((resolve, reject) => {
   img.crossOrigin = '';
   img.onload = () => {
    img.width = img.naturalWidth;
    img.height = img.naturalHeight;

    const datasetBytesBuffer =
       new ArrayBuffer(NUM_DATASET_ELEMENTS * IMAGE_SIZE * 4);

    const chunkSize = 5000;
    canvas.width = img.width;
    canvas.height = chunkSize;

    for (let i = 0; i < NUM_DATASET_ELEMENTS / chunkSize; i++) {
     const datasetBytesView = new Float32Array(
        datasetBytesBuffer, i * IMAGE_SIZE * chunkSize * 4,
        IMAGE_SIZE * chunkSize);
     ctx.drawImage(
        img, 0, i * chunkSize, img.width, chunkSize, 0, 0, img.width,
        chunkSize);

     const imageData = ctx.getImageData(0, 0, canvas.width, canvas.height);

     for (let j = 0; j < imageData.data.length / 4; j++) {
      // All channels hold an equal value since the image is grayscale, so
      // just read the red channel.
      datasetBytesView[j] = imageData.data[j * 4] / 255;
     }
    }
    this.datasetImages = new Float32Array(datasetBytesBuffer);

    resolve();
   };
   img.src = MNIST_IMAGES_SPRITE_PATH;
  });
```

```
  const labelsRequest = fetch(MNIST_LABELS_PATH);
  const [imgResponse, labelsResponse] =
      await Promise.all([imgRequest, labelsRequest]);

  this.datasetLabels = new Uint8Array(await labelsResponse.arrayBuffer());

  // Create shuffled indices into the train/test set for when we select a
  // random dataset element for training / validation.
  this.trainIndices = tf.util.createShuffledIndices(NUM_TRAIN_ELEMENTS);
  this.testIndices = tf.util.createShuffledIndices(NUM_TEST_ELEMENTS);

  // Slice the the images and labels into train and test sets.
  this.trainImages =
      this.datasetImages.slice(0, IMAGE_SIZE * NUM_TRAIN_ELEMENTS);
  this.testImages = this.datasetImages.slice(IMAGE_SIZE * NUM_TRAIN_ELEMENTS);
  this.trainLabels =
      this.datasetLabels.slice(0, NUM_CLASSES * NUM_TRAIN_ELEMENTS);
  this.testLabels =
      this.datasetLabels.slice(NUM_CLASSES * NUM_TRAIN_ELEMENTS);
}

nextTrainBatch(batchSize) {
  return this.nextBatch(
      batchSize, [this.trainImages, this.trainLabels], () => {
        this.shuffledTrainIndex =
            (this.shuffledTrainIndex + 1) % this.trainIndices.length;
        return this.trainIndices[this.shuffledTrainIndex];
      });
}

nextTestBatch(batchSize) {
  return this.nextBatch(batchSize, [this.testImages, this.testLabels], () => {
    this.shuffledTestIndex =
        (this.shuffledTestIndex + 1) % this.testIndices.length;
    return this.testIndices[this.shuffledTestIndex];
  });
}

nextBatch(batchSize, data, index) {
  const batchImagesArray = new Float32Array(batchSize * IMAGE_SIZE);
  const batchLabelsArray = new Uint8Array(batchSize * NUM_CLASSES);

  for (let i = 0; i < batchSize; i++) {
    const idx = index();

    const image =
        data[0].slice(idx * IMAGE_SIZE, idx * IMAGE_SIZE + IMAGE_SIZE);
    batchImagesArray.set(image, i * IMAGE_SIZE);

    const label =
```

```
        data[1].slice(idx * NUM_CLASSES, idx * NUM_CLASSES + NUM_CLASSES);
      batchLabelsArray.set(label, i * NUM_CLASSES);
    }

    const xs = tf.tensor2d(batchImagesArray, [batchSize, IMAGE_SIZE]);
    const labels = tf.tensor2d(batchLabelsArray, [batchSize, NUM_CLASSES]);

    return {xs, labels};
  }
}
```

Yes!
In TensorFlow.js, you **can save and load a model** very easily — even **in the browser's local storage**.

Here's the idea for your case:

- When the page loads, first try to **load** the model from `localstorage://my-model-name`.
- If it **doesn't exist**, then **create, train, and save** it.

Here's **how you can modify your `run` function** to do that:

```
javascript

async function run() {
  data = new MnistData();
  await data.load();

  // Try to load the model from local storage
  try {
    model = await tf.loadLayersModel('localstorage://my-mnist-model');
    console.log('Model loaded from storage.');
  } catch (error) {
    console.log('No saved model found. Creating a new one...');
    model = await getModel();
    await trainModel(model, data);

    // Save the model to local storage after training
    await model.save('localstorage://my-mnist-model');
    console.log('Model trained and saved.');
  }

  clearCanvas();
}
```

✅ **Summary of changes**:

- `tf.loadLayersModel('localstorage://my-mnist-model')` tries to load the saved model.
- If **successful**, you skip training.
- If it **fails** (no model found), you **train** the model and **save** it by `model.save('localstorage://my-mnist-model')`.

Important:

- If you want to **retrain from scratch** anytime, you'll need to `tf.io.removeModel('localstorage://my-mnist-model')` manually.

- You can also save to `indexeddb://` instead of `localstorage://` if you want more storage space.

Would you also like me to show you how to **add a button to delete/reset** the saved model if you want to retrain manually? 🚀
(very easy to add!)

> i use localhost:8000

Got it — you're serving your app on `localhost:8000`.

Now, to answer your **main question**:

> **Can I save the trained model, and load it if it exists without training again (only train if no saved model)?**

✅ Yes! TensorFlow.js (`tfjs`) allows you to **save** and **load** models in the browser, using **IndexedDB** (local database inside browser).

Here's **how you can modify your code**:

Step 1: Save and Load the Model

You can **save** and **load** using `tf.io.browserIndexedDB`:

```
javascript

async function getOrCreateModel() {
  const modelName = 'local-mnist-model'; // Unique name for storage

  try {
    // Try to load the model
    const model = await tf.loadLayersModel('indexeddb://' + modelName);
    console.log('✅ Loaded existing model from IndexedDB.');
    return model;
  } catch (error) {
    console.log('ℹ️ No saved model found. Creating and training a new one.');

    // No model found, create and train new model
    const model = await getModel();

    data = new MnistData();
    await data.load();

    await trainModel(model, data);

    // Save model after training
    await model.save('indexeddb://' + modelName);
    console.log('✅ Model saved to IndexedDB.');

    return model;
  }
}
```

Step 2: Update your `run` function

Instead of this:

```
javascript

async function run() {
  data = new MnistData();
  await data.load();
  model = await getModel();
  await trainModel(model, data);

  clearCanvas();
}
```

You do:

```
javascript

async function run() {
  model = await getOrCreateModel();
  clearCanvas();
}
```

(Notice: no re-training unless needed!)

Step 3: Things to Know

- When the user **refreshes** the page, **IndexedDB** persists, so the model will stay saved.
- If you **clear browser cache or storage**, the saved model will be lost.
- If you want to **force re-training**, you can manually delete the model using:

```
javascript

await tf.io.removeModel('indexeddb://local-mnist-model');
```

Quick Overview of Storage Methods

Save location	Example URL
IndexedDB (browser)	`indexeddb://model-name`
Local file (Node.js)	`file://path/to/model`
HTTP request	`http://your-server/model.json`

Final Code Idea

```
javascript

let model;
let data;

document.addEventListener('DOMContentLoaded', run);

async function run() {
  model = await getOrCreateModel();
```

```
  clearCanvas();
}
```

Would you like me to also show you a version that **downloads the model** as files (`.json` + `.bin`) if you want even more permanent storage (outside browser)? 🚀
(Useful for backup or server-side hosting later.)

i want save and load from my disk when i use local like i serve static files like js html

Perfect — you want to **save the trained model to your own disk** and **load it from disk** later, because you are serving files locally (`localhost:8000`) like a real app.
Not just saving temporarily in the browser (IndexedDB).

✅ Totally possible with **TensorFlow.js**!

Here's the idea:

- When training finishes ➔ `model.save('downloads://model-name')`
- This **downloads** two files:
  - `model-name.json`
  - `model-name.weights.bin`
- Then you **move them** into your `public` folder (where you serve static files like `.html`, `.js`).
- Later, you can **load the model** by URL:
  `tf.loadLayersModel('/model-name.json')`

Full Step-by-Step

1. Save the model

After training, you save it **to download**, like:

```
javascript

await model.save('downloads://mnist-model');
```

This will automatically download:

- `mnist-model.json`
- `mnist-model.weights.bin`

👉 Move these files into your server folder, e.g., `/public/models/`

---

2. Load the model from your server

Later, instead of creating a new model every time, **you load it from disk**:

```
javascript

const model = await tf.loadLayersModel('/models/mnist-model.json');
```

(assuming `/models/` is where you put it).

---

3. Automatic logic (load if exists, train if not)

You can try to load it first — if 404 (not found), **train and save**.

Example:

```
javascript

async function getOrCreateModel() {
  try {
    const model = await tf.loadLayersModel('/models/mnist-model.json');
    console.log('✅ Loaded model from disk.');
    return model;
  } catch (error) {
    console.log('ℹ️ Model not found, creating and training new one.');

    const model = await getModel();
    data = new MnistData();
    await data.load();
    await trainModel(model, data);

    // Save for user to download
    await model.save('downloads://mnist-model');
    console.log('✅ Model saved, please move it manually to /models/ folder.');

    return model;
  }
}
```

---

4. Your file structure after saving

```
pgsql

/public/
  index.html
  main.js
  /models/
    mnist-model.json
    mnist-model.weights.bin
```

Important Notes

- TensorFlow.js **cannot automatically save** to server folders from browser-side JavaScript because of browser security (CORS, filesystem access).
- It **downloads** files, and you **move them manually** into `/models/`.
- Once you move them, the app will **always load** without retraining.

Short Summary

Action	How it works
Save model	`` `model.save('downloads://model-name')` `` (user downloads files)
Load model	`` `tf.loadLayersModel('/models/model-name.json')` `` (from your server)
Move files	You move downloaded files manually into your project (e.g., `` `/models/` ``)

Would you like me to also show you how to **automatically trigger file moving** if you're using a Node.js server (instead of moving files manually)? 🚀
(Only possible if you control backend.)

so how should it look like to save and load from disk (if it is exists dont train - load saved model. if not exists - train) import { MnistData, NUM_TEST_ELEMENTS, NUM_TRAIN_ELEMENTS } from './data.js';

```
let model;
let data;

// Drawing
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');
let isDrawing = false;

// Setup canvas correctly for MNIST
canvas.width = 280;
canvas.height = 280;
canvas.style.width = '280px';
canvas.style.height = '280px';
canvas.style.imageRendering = 'pixelated';

ctx.fillStyle = 'black'; // 🛠️ background BLACK like MNIST
ctx.fillRect(0, 0, canvas.width, canvas.height);

canvas.addEventListener('mousedown', () => isDrawing = true);
canvas.addEventListener('mouseup', () => {
  isDrawing = false;
  ctx.beginPath();
});
canvas.addEventListener('mouseout', () => {
  isDrawing = false;
  ctx.beginPath();
});
canvas.addEventListener('mousemove', draw);

function draw(e) {
  if (!isDrawing) return;
  ctx.lineWidth = 20; // 🛠️ thicker lines, like MNIST digits
```

```
  ctx.lineCap = 'round';
  ctx.strokeStyle = 'white'; // 🛠️ draw white on black

  const rect = canvas.getBoundingClientRect();
  const scaleX = canvas.width / rect.width;
  const scaleY = canvas.height / rect.height;

  ctx.lineTo((e.clientX - rect.left) * scaleX, (e.clientY - rect.top) * scaleY);
  ctx.stroke();
  ctx.beginPath();
  ctx.moveTo((e.clientX - rect.left) * scaleX, (e.clientY - rect.top) * scaleY);
}

document.getElementById('clearBtn').addEventListener('click', clearCanvas);
document.getElementById('predictBtn').addEventListener('click', predictCanvas);

function clearCanvas() {
  ctx.fillStyle = 'black'; // 🛠️ black background
  ctx.fillRect(0, 0, canvas.width, canvas.height);
  ctx.beginPath();
}

// Model
async function getModel() {
  const model = tf.sequential();
  const IMAGE_WIDTH = 28;
  const IMAGE_HEIGHT = 28;
  const IMAGE_CHANNELS = 1;

  model.add(tf.layers.conv2d({
    inputShape: [IMAGE_WIDTH, IMAGE_HEIGHT, IMAGE_CHANNELS],
    kernelSize: 5,
    filters: 32,   // was 8
    strides: 1,
    activation: 'relu',
    kernelInitializer: 'varianceScaling'
  }));

  model.add(tf.layers.maxPooling2d({ poolSize: [2, 2], strides: [2, 2] }));

  model.add(tf.layers.conv2d({
    kernelSize: 5,
    filters: 64,   // was 16
    strides: 1,
    activation: 'relu',
    kernelInitializer: 'varianceScaling'
  }));

  model.add(tf.layers.maxPooling2d({ poolSize: [2, 2], strides: [2, 2] }));

  model.add(tf.layers.flatten());
```

```
  model.add(tf.layers.dense({   // new hidden dense layer
    units: 64,
    activation: 'relu',
    kernelInitializer: 'varianceScaling'
  }));

  model.add(tf.layers.dense({
    units: 10,
    activation: 'softmax',
    kernelInitializer: 'varianceScaling'
  }));

  const optimizer = tf.train.adam();
  model.compile({
    optimizer: optimizer,
    loss: 'categoricalCrossentropy',
    metrics: ['accuracy'],
  });

  return model;
}


async function trainModel(model, data) {
  const BATCH_SIZE = 128;
  const TRAIN_DATA_SIZE = NUM_TRAIN_ELEMENTS ;
  const TEST_DATA_SIZE = NUM_TEST_ELEMENTS;

  const [trainXs, trainYs] = tf.tidy(() => {
    const d = data.nextTrainBatch(TRAIN_DATA_SIZE);
    return [
      d.xs.reshape([TRAIN_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  const [testXs, testYs] = tf.tidy(() => {
    const d = data.nextTestBatch(TEST_DATA_SIZE);
    return [
      d.xs.reshape([TEST_DATA_SIZE, 28, 28, 1]),
      d.labels
    ];
  });

  return model.fit(trainXs, trainYs, {
    batchSize: BATCH_SIZE,
    validationData: [testXs, testYs],
    epochs: 10,
    shuffle: true,
    callbacks: tfvis.show.fitCallbacks(
```

```
      { name: 'Training Performance' },
      ['loss', 'val_loss', 'acc', 'val_acc'],
      { height: 200, callbacks: ['onEpochEnd'] }
    )
  });
}

async function predictCanvas() {
  // Shrink big canvas down to 28x28
  const smallCanvas = document.createElement('canvas');
  smallCanvas.width = 28;
  smallCanvas.height = 28;
  const smallCtx = smallCanvas.getContext('2d');

  smallCtx.drawImage(canvas, 0, 0, 28, 28);

  const imgData = smallCtx.getImageData(0, 0, 28, 28);
  const data = imgData.data;

  const grayData = new Float32Array(28 * 28);

  for (let i = 0; i < data.length; i += 4) {
    // Remember: after draw, white pixel means drawn, black means background
    const r = data[i];
    const g = data[i + 1];
    const b = data[i + 2];
    const avg = (r + g + b) / 3;
    // Normalize to [0,1], background = 0, digit = 1
    grayData[i / 4] = avg / 255;
  }

  const input = tf.tensor4d(grayData, [1, 28, 28, 1]);

  const prediction = model.predict(input);
  const pred = prediction.argMax(1);
  const result = (await pred.data())[0];

  document.getElementById('prediction').innerText = `Prediction: ${result}`;

  // Dispose tensors
  input.dispose();
  prediction.dispose();
  pred.dispose();
}

// Run everything
async function run() {
  data = new MnistData();
  await data.load();
  model = await getModel();
  await trainModel(model, data);
```

```
  clearCanvas();
}

document.addEventListener('DOMContentLoaded', run);
```

Good — your code is **already excellent**.
Now we just need to **adjust `run()`** so it:

- tries to **load model from server** if it exists
- if it **fails** (404), it will **train a new model and allow you to save**

Here is the **full correct version of `run()`** you should use:

```javascript

async function run() {
  data = new MnistData();
  await data.load();

  try {
    // Try loading model from your server (models/ directory)
    model = await tf.loadLayersModel('/models/mnist-model.json');
    console.log('✅ Loaded saved model from disk.');
  } catch (error) {
    console.warn('⚠ Could not load model, training a new one...');

    // Create new model
    model = await getModel();

    // Train new model
    await trainModel(model, data);

    // Save model to download
    await model.save('downloads://mnist-model');

    console.log('✅ Model trained and downloaded. Move it to /models/ folder manually!');
  }

  clearCanvas();
}
```

✅ **EXPLANATION:**

- First it **loads** `/models/mnist-model.json` from your server.
- If it **succeeds**, it's ready, no training needed.
- If it **fails** (for example, because the files don't exist), it **trains new model** and **downloads** it.
- You **manually move** `mnist-model.json` and `mnist-model.weights.bin` to `/public/models/` or wherever your server serves static files.

Your folder structure should look like:

```
pgsql

/public/
  index.html
  main.js
  /models/
    mnist-model.json
    mnist-model.weights.bin
```

Also update this line in your HTML

Make sure you have correct paths in your `<script>` tags:

```
html

<script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs"></script>
<script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs-vis"></script>
<script src="./data.js"></script>
<script src="./main.js"></script> <!-- your file with the code -->
```

✅ Final flow

Step	What happens
First time (no model yet)	Trains, downloads model files
You manually put files in `/models/`	
Next time	Loads from disk immediately — no retraining

Would you also like me to show you how to **skip manual moving** automatically if you use a local Node.js + Express server? 🚀 (Very easy if you want!)
Or would you prefer to keep it manual for now?

ok but will saving and loading work when i deploy app on render?

Good question. Short answer:

✅ Yes — but only if you save your model *before* deployment and put it into your static assets.

🚀 Why?

When you deploy to a service like **Render**, **Vercel**, or **Netlify**, you can't "write" files permanently during runtime —
because the server filesystem is **read-only after deploy** (for static sites).

So:

- You **train locally** once.
- You **download** the files (`mnist-model.json`, `mnist-model.weights.bin`).

- You **put them into your project folder** (e.g., `/public/models/`).
- When you deploy, **Render uploads those files** with your app.

Then your app can **read** them normally at `/models/mnist-model.json`. ✅

🔥 Important:

During deployment, your app **cannot call**:

```
javascript

await model.save('downloads://mnist-model');
```

because in production **you cannot download to the server** — only to the user.

That's why we train **locally** before deployment, **save** manually, and then **deploy with the model already existing**.

📦 Your final app structure before deploying:

```
pgsql

/public/
  index.html
  main.js
  data.js
  /models/
    mnist-model.json
    mnist-model.weights.bin
```

and you serve `/public` folder statically.

💬 In short:

Local	Production (Render, etc.)
First time: train if no model.	No training. Only load.
Save model after training.	Model must already be uploaded with project.

✅ In production

This code will **only** load:

```
javascript

model = await tf.loadLayersModel('/models/mnist-model.json');
```

and **never** train.

Would you like me to also show you **how to automatically skip training** based on environment (development vs production)?

It would make your `run()` even smarter if you want! 🚀
(only 2-3 lines extra)

so check me if i understand correctly. i have to deploy all files with some ready model, and then i can save and load. if so give me correct code

Exactly — you understood it **correctly**! 👍

👉 You need to **train and save your model locally first** —
then **deploy the saved model files together** with your project.
After that, on the deployed app (e.g., on Render), you **only load**, never train.

🛠️ Correct Code You Should Use

Here is the **correct `run()` function** you need:

```
javascript

async function run() {
  data = new MnistData();
  await data.load();

  // Try to load the model
  try {
    model = await tf.loadLayersModel('/models/mnist-model.json');
    console.log('✅ Loaded pre-trained model.');
  } catch (error) {
    console.log('⚠ No saved model found. Training new model...');
    model = await getModel();
    await trainModel(model, data);

    // Save model locally after training (only when running locally)
    await model.save('downloads://mnist-model');
    console.log('✅ Model trained and saved.');
  }

  clearCanvas();
}
```

✍️ How it works:

1. **Try to load `mnist-model.json`** from **`/models/`**.
2. **If found →** skip training.
3. **If not found → train**, then **save** the model to your local computer (downloads).
4. **You manually move** the downloaded files (**`mnist-model.json`** and **`mnist-model.weights.bin`**) into **`/public/models/`**.

Then **deploy everything** to Render!

📂 Directory structure you should have before deploying:

```
pgsql

/public/
  index.html
  main.js
  data.js
  /models/
    mnist-model.json
    mnist-model.weights.bin
```

(You can also call it `/static/` if your server expects it — depends on your setup.)

❗ Important notes:

- In **local** mode: the app trains and downloads the model automatically.
- In **production** (Render): the model already exists in `/models/`, so no training happens anymore.
- **Never use `model.save()` to server** during deployed runtime — browser will still download it locally if user triggers it.
- Always **deploy the model files** with your static assets.

✅ Final summary:

Situation	What happens
Localhost, no model	Trains a new model and saves it.
Localhost, model exists	Loads model, no training needed.
Render deployed app	Loads model, no training allowed.

Would you also like me to show you a tiny **optimization** so you don't try `await data.load()` if model loading succeeds? (cleaner and faster 🧹)
I can improve your final code if you want! 🚀

why diffrent paths model = await tf.loadLayersModel('/models/mnist-model.json'); await model.save('downloads://mnist-model');

Good question — you're thinking **very precisely** — let me explain **clearly**:

Line	Meaning	Where it saves/loads
`await tf.loadLayersModel('/models/mnist-model.json')`	**Load** a model **from server folder** `/models/`	**Server** (static file like HTML/JS)
`await model.save('downloads://mnist-model')`	**Save** a model **to your local computer** as download	**User's browser downloads** the files

---

🧠 Why are they different?

- `loadLayersModel('/models/xxx')` is for **loading** *from server* (your deployed app's files).

- `save('downloads://xxx')` is for **saving** *to user's own computer*.

They are **two different protocols**:

Protocol	Purpose
`/models/` (HTTP path)	Get files from server (like loading `.js`, `.html`)
`downloads://` (browser special path)	Tell browser to **download** files

Saving in browser **cannot** directly put files back to server!
JavaScript running in browser **has no permission** to write on server disk.

That's why:
🔵 **You save locally** → 🔵 **You manually upload** `mnist-model.json` and `mnist-model.weights.bin` into `/models/` folder before deploy.

🛠️ Correct Workflow

1. **First time** on localhost:
   - Model not found → train
   - `save('downloads://mnist-model')` pops up download → you save the files.
2. **You manually move** saved files into `/models/` folder in your project.
3. **Deploy** project (with `/models/` inside).
4. **Now tf.loadLayersModel('/models/mnist-model.json')** will find it immediately → no more training needed.

🔥 In short:

- `save('downloads://')` ➔ **saves to your computer** (browser download)
- `load('/models/xxx')` ➔ **loads from your deployed project files** (server fetch)

If you want,
I can even show you **how to save directly inside IndexedDB** (inside browser) without download popup — fully automatic — but that's more advanced. 🚀

Would you like me to show that too? (Optional)

> but i want have possibility to save new model on render

Got it — you want **saving the new model dynamically** *while your app runs on Render (deployed)*.
✅ **You want users (or you) to train and save new models**.

⚡ **Important fact**:
When your app is deployed on **Render** (or Vercel, Netlify, etc.),
your app is usually **static** → **JavaScript in browser cannot write to server disk directly**.

Browsers cannot save directly to server file system.
That is **blocked for security reasons**.

🛠️ So how can we really save a model "online"?

You need **a backend API** to accept model files.

Solution:

1. **After training**, you **save model files into memory**.
2. You **send (POST)** model files to your server backend.
3. Your backend (Node.js, Python, whatever) **saves them on disk**.

🔥 Practical steps:

1. Frontend (after training)

```
javascript

await model.save(tf.io.browserHTTPRequest('/upload-model', { method: 'POST' }));
```

✅ This will **POST** the model files to `/upload-model`.

---

2. Backend (Node.js example using Express)

```
javascript

import express from 'express';
import multer from 'multer';
import fs from 'fs';

const app = express();
const upload = multer({ dest: 'uploads/' }); // Temporary upload folder

app.post('/upload-model', upload.any(), (req, res) => {
  if (!fs.existsSync('./models')) {
    fs.mkdirSync('./models');
  }

  for (const file of req.files) {
    if (file.originalname === 'model.json') {
      fs.renameSync(file.path, './models/mnist-model.json');
    } else if (file.originalname.endsWith('.bin')) {
      fs.renameSync(file.path, './models/mnist-model.weights.bin');
    }
  }

  res.send('Model uploaded and saved.');
});

app.use(express.static('public')); // Your frontend files (HTML, JS)
app.listen(8000, () => console.log('Server running on http://localhost:8000'));
```

✅ This server:

- Accepts `POST /upload-model`
- Saves `mnist-model.json` and `mnist-model.weights.bin` into `/models/`
- Then you can later `tf.loadLayersModel('/models/mnist-model.json')`.

📦 Summary:

Without backend	With backend
Only download to user PC	Uploads and saves to server
No server write	Server accepts POST and writes

⚡ Final corrected code for your frontend:

```
javascript

// When model needs saving
await model.save(tf.io.browserHTTPRequest('/upload-model', { method: 'POST' }));
```

🧠 So:

- If you want just "download model" → `downloads://`
- If you want "save new model on server" → you need an **upload endpoint** like `/upload-model`.

✅ Short answer to your goal:

> **Yes, you can save a new model on Render, but you must implement an API endpoint to receive it.**

Would you like me to give you a full working ready-to-upload-to-Render project example (`server.js`, `frontend.html`) too? 🚀
(so you just clone, deploy, and it works)
👉 **It'll cost maybe 3-4 min to show.** Want it?

how to cancel git from save_model PS C:\PROJECT_HANDWRITING> git add .
warning: adding embedded git repository: save_model
hint: You've added another git repository inside your current repository.
hint: Clones of the outer repository will not contain the contents ofhint: the embedded repository
and will not know how to obtain it.
hint: If you meant to add a submodule, use:
hint:
hint: git submodule add <url> save_model
hint: If you added this path by mistake, you can remove it from the

```
hint: index with:
hint:
hint:   git rm --cached save_model
hint: Disable this message with "git config set advice.addEmbeddedRepo false"
PS C:\PROJECT_HANDWRITING> git rm --cached save_model
error: the following file has staged content different from both the
file and the HEAD:
    save_model
(use -f to force removal)
PS C:\PROJECT_HANDWRITING> cd save_model
PS C:\PROJECT_HANDWRITING\save_model> git rm --cached save_model
fatal: pathspec 'save_model' did not match any files
```

Alright, you're very close. Here's what's happening and exactly how to fix it cleanly:

🛠️ What's happening:

You accidentally **added a `.git/` inside `save_model/`**,
which makes Git think `save_model/` is a **separate repository** (an embedded Git repo — *submodule*).
That's why you get this message about *embedded repository*.

You **don't want that** → you just want `save_model/` to be **normal folder**, not a repo inside a repo.

✅ Correct steps to fix:

1. **Delete `.git` inside `save_model/`** folder (this removes its repo history):

```
bash

PS C:\PROJECT_HANDWRITING\save_model> rm -r -force .git
```

or in pure cmd:

```
cmd

rmdir /s /q .git
```

✅ Now `save_model/` is **just a regular folder**.

---

2. **Go back to main project folder** and tell Git to **remove `save_model/` from index** (cached):

```
bash

PS C:\PROJECT_HANDWRITING> git rm --cached -r save_model
```

✅ This removes it *from Git staging*, but **not deletes the folder** from disk.

---

3. **(Optional)** Add it properly again:

```
bash

git add save_model
```

Now Git will treat `save_model/` like normal files (no submodule issues).

---

4. **Commit the fix:**

```
bash

git commit -m "Fix: remove accidental git repo inside save_model"
```

🧠 Why this happens:

When you `tf.save` model sometimes, if you did it weirdly or copied from another repo, it can accidentally contain `.git/` folder inside.

Git **does not like repos inside repos** unless you manually make them a **submodule** — but you don't want that here.

⚡ In short:

Command	Purpose
`rm -r -force .git` inside `save_model/`	Removes accidental git repo inside the folder
`git rm --cached -r save_model`	Unstage it
`git add save_model`	Add it cleanly
`git commit`	Save changes

Would you also like me to show you how to **ignore future model saves automatically** with `.gitignore` (like: don't track temp model saves)? 🚀
Might be a good idea if you're training often!

```
PS C:\PROJECT_HANDWRITING\save_model> rm -r -force .git
PS C:\PROJECT_HANDWRITING\save_model> cd..
PS C:\PROJECT_HANDWRITING> git rm --cached -r save_model
error: the following file has staged content different from both the
file and the HEAD:
    save_model
(use -f to force removal)
PS C:\PROJECT_HANDWRITING>
```

It looks like Git is still holding onto `save_model` in a state that's different from both your current working directory and the previous commit (because of staged changes). To remove it properly from Git staging, you'll need to **force the removal** of the file with the `-f` flag.

Here's how to do it:

1. **Force remove `save_model/` from the Git index** (this will unstage it, so Git stops tracking the folder):

```
bash

git rm -r --cached -f save_model
```

The `-f` flag forces the removal, even if the contents are different from the previous commit or are staged.

2. **Commit the removal**:

```
bash

git commit -m "Remove save_model folder from Git tracking"
```

Optional: If you want to ignore future changes in `save_model/` (to prevent Git from tracking the model files):

Create or update the `.gitignore` file in the root of your project with:

```
save_model/
```

This will ensure that `save_model/` and its contents are ignored by Git in future commits.

After doing this, you should no longer see the warnings or errors related to `save_model`. Let me know if you encounter any issues!

```
PS C:\PROJECT_HANDWRITING\nac> git rm -r --cached -f nac
fatal: pathspec 'nac' did not match any files
```

The error message `"fatal: pathspec 'nac' did not match any files"` means that Git cannot find a directory or file named `nac` in the current directory, so it cannot remove it from the index.

Here are some things you can check and try:

1. **Check the exact name of the directory or file**: Use the `ls` (on Unix-based systems) or `dir` (on Windows) command to list the files and directories in your current folder. Ensure that `nac` is spelled correctly and exists in the directory you're working from.

   For example, try:

   ```
   bash

   dir
   ```

2. **Check Git's current tracked files**: Use `git status` to see if the `nac` folder is currently being tracked by Git. It will show you which files and folders are staged, modified, or untracked.

   ```
   bash

   git status
   ```

   If `nac` is listed in the staged changes, but the `git rm` command fails, you can also try running:

   ```
   bash

   git rm -r --cached nac
   ```

3. **Ensure you're in the right directory**: Ensure that you're in the correct directory where `nac` exists (i.e., the root directory of your Git repository). If you're unsure, you can check the path with:

   ```
   bash

   pwd  # on Linux/macOS
   cd   # on Windows
   ```

4. **Force removal with full path**: If the above steps don't work, you can try specifying the full path to `nac` relative to the root of your Git repository. For example:

   ```
   bash

   git rm -r --cached ./nac
   ```

Once you successfully remove the folder from Git's tracking, proceed to commit the changes:

```
bash

git commit -m "Remove nac folder from Git tracking"
```

Let me know how it goes!

PS C:\PROJECT_HANDWRITING> git rm -r --cached -f nac
rm 'nac'
PS C:\PROJECT_HANDWRITING> git add .
warning: adding embedded git repository: nac
hint: You've added another git repository inside your current repository.
hint: Clones of the outer repository will not contain the contents ofhint: the embedded repository
and will not know how to obtain it.
hint: If you meant to add a submodule, use:
hint:
hint: git submodule add <url> nac
hint:
hint: If you added this path by mistake, you can remove it from the
hint: index with:

```
hint:
hint:   git rm --cached nac
hint:
hint: See "git help submodule" for more information.
hint: Disable this message with "git config set advice.addEmbeddedRepo false"
```

The warning you're seeing indicates that Git is detecting an embedded Git repository inside your project directory. This means that the `nac` directory is also a Git repository itself, possibly because there is a `.git` folder inside it. Git is treating it as a submodule, which is not what you intended.

To resolve this issue, follow these steps:

1. Remove the Embedded Git Repository:

If you do not want the `nac` directory to be treated as a submodule and simply want to remove it from your current Git repository, do the following:

- First, ensure the `.git` directory inside the `nac` folder is removed:

```bash
rm -r nac/.git
```

- Then, add `nac` back to the staging area:

```bash
git add .
```

2. Commit the Change:

Once you've removed the `.git` folder from inside `nac`, you can commit the changes:

```bash
git commit -m "Remove embedded Git repository (nac)"
```

3. Check the Status:

After removing the `.git` folder and committing, check the status to ensure everything is tracked properly:

```bash
git status
```

This should fix the issue of Git treating the `nac` folder as an embedded repository. Let me know if you encounter any more issues!

```
fatal: pathspec 'nac' did not match any files
PS C:\PROJECT_HANDWRITING> git rm -r --cached -f nac
PS C:\PROJECT_HANDWRITING> git rm -r --cached -f nac2
rm 'nac2'
PS C:\PROJECT_HANDWRITING> git rm -r --cached -f nac2k
rm 'nac2k'
PS C:\PROJECT_HANDWRITING> git rm -r --cached -f nack
rm 'nack'
PS C:\PROJECT_HANDWRITING> git init
Reinitialized existing Git repository in C:/PROJECT_HANDWRITING/.git/PS
C:\PROJECT_HANDWRITING> git add .
warning: in the working copy of 'nac/.gitignore', LF will be replaced by CRLF the next time Git
touches it
warning: in the working copy of 'nac/.gitignore copy', LF will be replaced by CRLF the next time Git
touches it
warning: adding embedded git repository: nac2
hint: You've added another git repository inside your current repository.
hint: Clones of the outer repository will not contain the contents ofhint: the embedded repository
and will not know how to obtain it.
hint: If you meant to add a submodule, use:
hint:
hint:   git submodule add <url> nac2
hint:
hint: If you added this path by mistake, you can remove it from the
hint: index with:
hint:
hint:   git rm --cached nac2
hint:
hint: See "git help submodule" for more information.
hint: Disable this message with "git config set advice.addEmbeddedRepo false"
warning: adding embedded git repository: nac2k
warning: adding embedded git repository: nack
```

It seems like the **`nac2`**, **`nac2k`**, and **`nack`** directories are still being recognized as embedded Git repositories because they contain their own **`.git`** directories. You need to remove the **`.git`** directories from those folders before adding them to your current Git repository.

Here's how you can fix it:

1. **Remove the `.git` directories from the subdirectories:**

   Navigate to the directories (**`nac2`**, **`nac2k`**, and **`nack`**) and remove the **`.git`** folder inside each of them:

   ```bash
   rm -r nac2/.git
   rm -r nac2k/.git
   rm -r nack/.git
   ```

2. **Add the changes to Git:**

After you've removed the `.git` folders, go back to your main project folder and add the changes to Git:

```
bash

git add .
```

3. **Commit the changes:**

   Once the changes are staged, commit them to your Git repository:

```
bash

git commit -m "Removed embedded Git repositories (nac2, nac2k, nack)"
```

4. **Push the changes (if applicable):**

   If you're working with a remote repository, don't forget to push the changes:

```
bash

git push
```

This should stop Git from treating `nac2`, `nac2k`, and `nack` as submodules and solve the warnings about the embedded Git repositories.

how to remove git repo when PS C:\PROJECT_HANDWRITING\nac> git init
Initialized empty Git repository in C:/PROJECT_HANDWRITING/nac/.git/

If you want to remove a Git repository from a specific directory (in this case, the `nac` directory), you can simply delete the `.git` folder inside that directory. Here's how to do it:

1. **Navigate to the `nac` directory:**

   If you're already in the `nac` directory, you can skip this step. Otherwise, use the following command to go to the `nac` directory:

```
bash

cd C:\PROJECT_HANDWRITING\nac
```

2. **Remove the `.git` directory:**

   To completely remove the Git repository from the `nac` directory, delete the `.git` directory. This will uninitialize the Git repository in that folder:

```
bash

rm -r .git
```

3. **Verify that the `.git` directory has been removed:**

After running the above command, you can check if the `.git` directory has been successfully removed by running:

```
bash

ls -a
```

This will list all files in the directory, and you should no longer see the `.git` folder listed.

4. **Check the status:**

   You can also run the following Git command to check if the directory is still considered a Git repository:

```
bash

git status
```

   If the directory is no longer a Git repository, it will display an error saying that the directory is not a Git repository.

By removing the `.git` directory, you've effectively removed the Git repository from the `nac` directory, and the folder will no longer be tracked as a Git repository.

```
PS C:\PROJECT_HANDWRITING\nac> gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol https
✓ Configured git protocol
✓ Logged in as arturr0
! You were already logged in to this account
PS C:\PROJECT_HANDWRITING\nac> rm -r .git
>>
rm : Cannot remove item C:\PROJECT_HANDWRITING\nac\.git: You do not
have sufficient access rights to perform this operation.
At line:1 char:1
+ rm -r .git
+ ~~~~~~~~~~
    + CategoryInfo          : PermissionDenied: (C:\PROJECT_HANDWRI
   TING\nac\.git:DirectoryInfo) [Remove-Item], IOException
    + FullyQualifiedErrorId : RemoveFileSystemItemUnAuthorizedAcces
   s,Microsoft.PowerShell.Commands.RemoveItemCommand
```

The error you're seeing suggests that you don't have the necessary permissions to delete the `.git` directory. To resolve this, you can try the following steps:

1. Check Your Permissions

Make sure you have the correct permissions to modify or delete the `.git` folder.

- Right-click on the `C:\PROJECT_HANDWRITING\nac` folder and choose **Properties**.
- Go to the **Security** tab and check if your user account has full access to the folder.
- If not, click **Edit** and give your account full permissions.

2. Use Elevated PowerShell (Administrator Mode)

If you still have issues with permissions, try running PowerShell with elevated (Administrator) privileges.

- Right-click on **Start** and select **Windows PowerShell (Admin)**.
- Navigate to the folder where your repository is located using `cd`:

```
powershell

cd C:\PROJECT_HANDWRITING\nac
```

- Now try deleting the `.git` directory again:

```
powershell

rm -r .git
```

3. Alternative Deletion Command

If PowerShell still refuses to delete the `.git` directory, you can try using the `del` command or `Remove-Item` in a different way:

- Open PowerShell as administrator.
- Navigate to the directory:

```
powershell

cd C:\PROJECT_HANDWRITING\nac
```

- Try removing the `.git` directory with:

```
powershell

Remove-Item -Recurse -Force .git
```

4. Check for Running Git Processes

Sometimes, the `.git` folder might be locked by an ongoing Git process. You can check if there are any Git processes running and stop them.

- Open **Task Manager** (Ctrl + Shift + Esc).
- Look for any processes related to Git, such as `**git.exe**`, and end them.
- Once the process is stopped, try removing the `**.git**` directory again.

5. Unlocking Files (if needed)

If some files in the `**.git**` directory are locked by other processes (like antivirus or file indexing), you can try using a tool like **Unlocker** or **LockHunter** to unlock the files before attempting to delete them again.

Let me know if you encounter any other issues during the process!